fixes from Jacobson's 1987 paper

1. round-trip-time variance estimation
2. exponential retransmit timer backoff
3. *slow-start*
4. more aggressive receiver ack policy
5. dynamic window sizing on congestion
6. Karn's clamped retransmit backoff
7. fast retransmit

“slow start”

not very well named

exponential(ish) increase to find window size

- used on connection setup or some losses

history: original TCP set high window size

slow start is “slow” because it starts at a small size always

… but it grows *really quickly*

“slow start”

set window size = $\sim$ 2 packet

increase by one packet for each ACK

… until first packet loss

use window size before first loss

slow start effect

suppose we never leave slow start in connection between A and B, with initial window size of 1 packet, and:

A sends 4 packets to B

after receiving 4 packets, B sends 8 packets to A

after receiving those packets A sends 1 packet to B

how many round-trip times does this take?

TCP Tahoe

congestion avoidance (cwnd $\geq$ ssthresh)

no loss: cwnd += 1 segment per round-trip-time

on loss: ssthresh = cwnd/2; cwnd = ~ 2 segments

"slow start" (cwnd < ssthresh)

no loss: cwnd += 1 segment per newly ack'd segment

on loss: ssthresh = cwnd/2; cwnd = cwnd/2;

recovery (known loss)

retransmit packet after third duplicate ACK of previous packet :or timeout

TCP (New)Reno [with slow start]

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

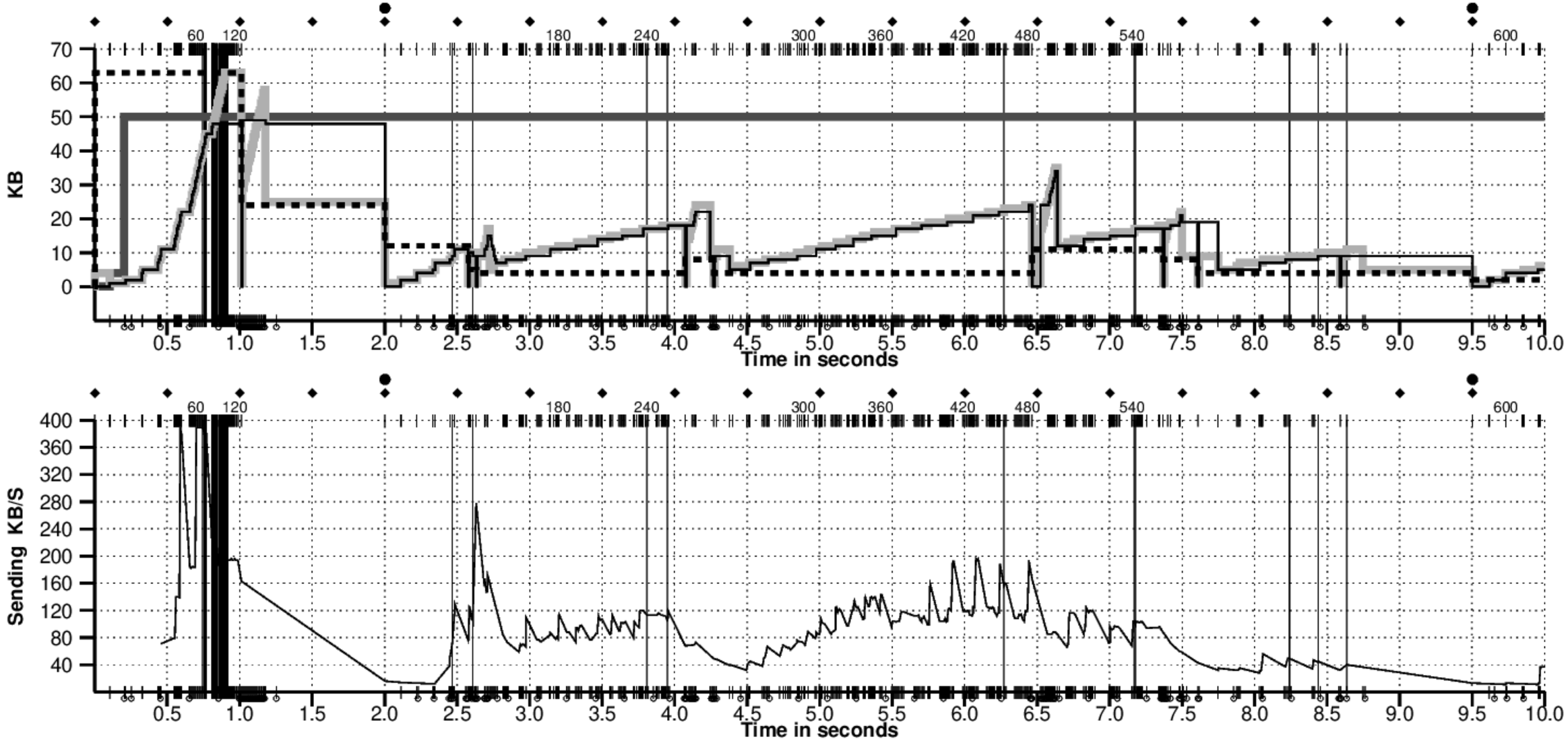
- no loss: cwnd += 1 segment per round-trip-time
- on loss (dup ACK): cwnd = cwnd/2; ssthresh = new cwnd
- on loss (timeout): cwnd = 2 segments

"slow start" (cwnd < ssthresh)

- *no loss: cwnd += 1 segment per newly ack'd segment*
- *on loss: ssthresh = cwnd/2; cwnd = cwnd/2;*

recovery (known loss)

- resend immediately on third dup ACK
- send new data based on self-clocking idea ("fast recovery")

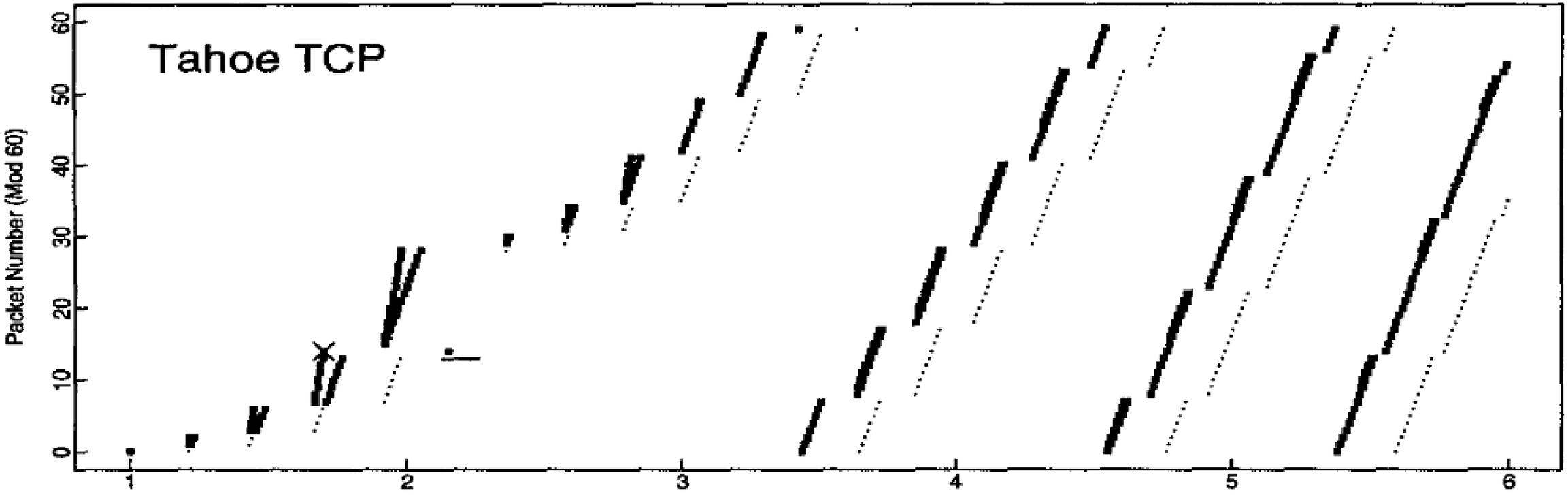


top thick, light-grey line = congestion window; dotted = slow start threshold

from Brakmo, O'Malley, and Peterson, "TCP Vegas: New techniques for congestion detection and avoidance"

TCP 'Tahoe' w/ one loss

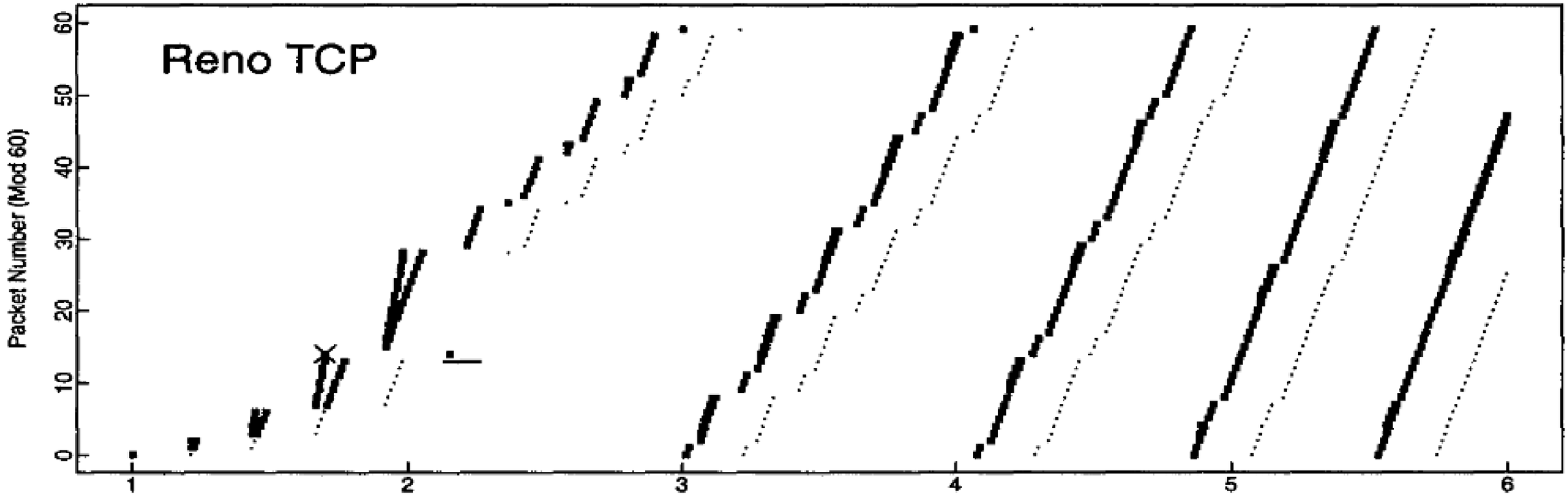
from Kevin Fall and Sally Floyd, "Simulation-based Comparisons of Tahoe, Reno, and SACK TCP"



TCP Tahoe = slow start, fast retransmit, no fast recovery

TCP 'Reno' w/ one loss

from Kevin Fall and Sally Floyd, "Simulation-based Comparisons of Tahoe, Reno, and SACK TCP"



TCP Reno = slow start, fast retransmit/recovery

fixes from Jacobson's 1987 paper

1. round-trip-time variance estimation
2. exponential retransmit timer backoff
3. slow-start
4. more aggressive receiver ack policy
5. dynamic window sizing on congestion
6. Karn's clamped retransmit backoff
7. *fast retransmit*

TCP (New)Reno [with slow start]

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

- no loss: cwnd += 1 segment per round-trip-time
- on loss (dup ACK): cwnd = cwnd/2; ssthresh = new cwnd
- on loss (timeout): cwnd = 2 segments

"slow start" (cwnd < ssthresh)

- no loss: cwnd += 1 segment per newly ack'd segment
- on loss: ssthresh = cwnd/2; cwnd = cwnd/2;

recovery (known loss)

- *resend immediately on third dup ACK*
- send new data based on self-clocking idea ("fast recovery")

fast retransmit

if large window + data packet 2 is lost, then sender will see

ACK 0, *ACK 1, ACK 1, ACK 1, ACK 1, ACK 1*

duplicate ACKs indicate missing packet 2

shouldn't wait for timeout

$\rightarrow$ TCP heuristic: retransmit immediately after ~ 3 duplicate ACKs

not 1 duplicate ACK to tolerate some reordering

also some other details (we'll talk later)

fast retransmission

TCP calls this idea of retransmission from duplicate ACKs "fast retransmission"

means we typically don't wait for timeouts

… assuming just one loss

fast retransmit delays

recv'd	sent
—	data 0-7
ACK 0	
	data 8
ACK 1	
	data 9
ACK 1	
ACK 1	
	data 2
ACK 1	
ACK 1	
ACK 1	
ACK 1	
ACK 1	
ACK 9	
	data 10-…

delayed sending new packets until ACK 9

… but probably could've done that earlier?

TCP (New)Reno [with slow start]

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

- no loss: cwnd += 1 segment per round-trip-time
- on loss (dup ACK): cwnd = cwnd/2; ssthresh = new cwnd
- on loss (timeout): cwnd = 2 segments

"slow start" (cwnd < ssthresh)

- no loss: cwnd += 1 segment per newly ack'd segment
- on loss: ssthresh = cwnd/2; cwnd = cwnd/2;

recovery (known loss)

- resend immediately on third dup ACK
- *send new data based on self-clocking idea ("fast recovery")*

normal packets in flight

recv'd	sent	count of packets in flight
—	data 0-7	8
ACK 0		7
	data 8	8
ACK 1		7
	data 9	8
ACK 2		7
	data 10	8
ACK 3		7
	data 11	8
ACK 4		7
	data 12	8

window size $\sim$ packets in flight normally

duplicate ACK waste

window size $\sim$ packets in flight normally

but duplicate ACKs are exception (say window size 8)

recv'd	sent	count of packets in flight
—	data 0-7	8
ACK 0		7
	data 8	8
ACK 1		7
	data 9	8
ACK 1		7
ACK 1		6
	data 2	5
ACK 1		4
ACK 1		3
ACK 1		2

problem: need to wait until ACK of 2 to make progress

alternate explanation

sender stopped sending while receiving duplicate ACKs

but we know *most messages got there*

means our usage of network doesn't reflect our window size

want to transmit new packets in response to these ACKs

- even though out of 'normal' window

TCP's fast recovery

estimate *packets in flight*

- with no losses, `cwnd` already does this!
- problem: needs adjustment there are losses

transmit new packets when packets in flight < cwnd

- two implementation ideas:
- temporarily 'inflate' cwnd
- track effective send window separately

fast recovery example

recv'd	sent	est. packets in flight	send window size (range)	cwnd = target packets in flight
—	data 0-7	8	8 (0-7)	8
ACK 0		7	8 (0-7)	8
	data 8	8	8 (1-8)	8
ACK 1		7	8 (1-8)	8
	data 9	8	8 (2-9)	8
ACK 1 (+3)		7	8 (2-9)	8
ACK 1 (+3-4)		6	8 (2-9)	8
ACK 1 (+3-5)		5	8 (2-9)	4
	data 2	6	8 (2-9)	4
ACK 1 (+3-6)		5	8 (2-9)	4
ACK 1 (+3-7)		4	8 (2-9)	4
ACK 1 (+3-8)		3	8 (2-9)	4
	data 10	4	9 (2-10)	4
ACK 1 (+3-9)		3	9 (2-10)	4
	data 11	4	10 (2-11)	4

during recovery, "inflate" effective send window size

try to make actual packets in flight match new congestion window size

selective ACKs

selective ACK: ACK 1, plus I got 3–5 and 7–8

can use as another duplicate ACK signal:

example:
resend 6 immediately after
3x “plus I got 3–5 and 7–…”

the reverse path

so far: assuming congestion on sender to receiver path

but we can also have congestion in other direction

- network becomes overloaded with ACKs

hopefully rare because ACKs are small, but…

but worth some special mitigations

delayed ACKs

RFC 1122 (Requirements for Internet Hosts — Communication Layers)

> "A host that is receiving a stream of TCP data segments can increase efficiency… by sending fewer than one ACK (acknowledgment) per data segment received; this is known as a "delayed ACK"… "

usually enabled these days

adds some latency, so Linux lets you disable on per-connection basis

diversion: some queuing theory

queuing theory: applied probability

talks about how queues work

applies to networks and anything else with “waiting in line”

queue measurements

arrival rate

service time (amount of time after waiting in line)

utilization = arrival rate / service time

if single thing can be processed at a time, then max utilization = 100%

- higher implies “infinitely” long queues

M/M/1/∞ queue

next slides: results for M/M/1/∞ queue

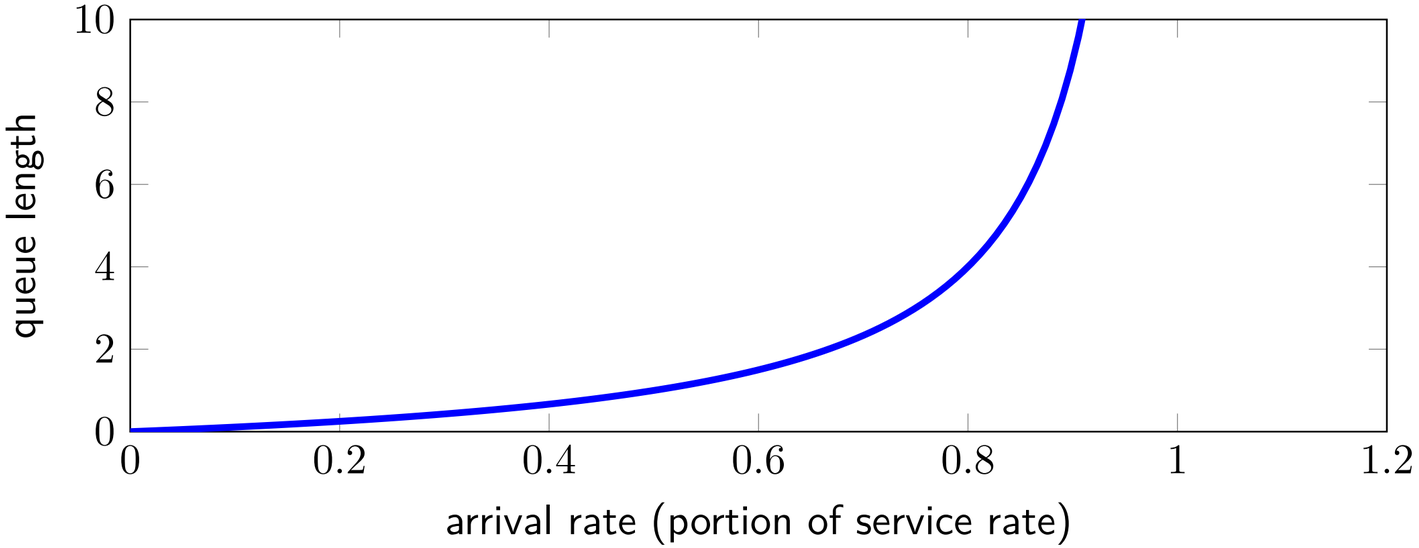
M (memoryless) — random arrival (exponential dist.)

M — random service time (exponential dist.)

1 — one “server” (thing that can process packets)

∞ — unlimited queue length

M/M/1/∞ queue length

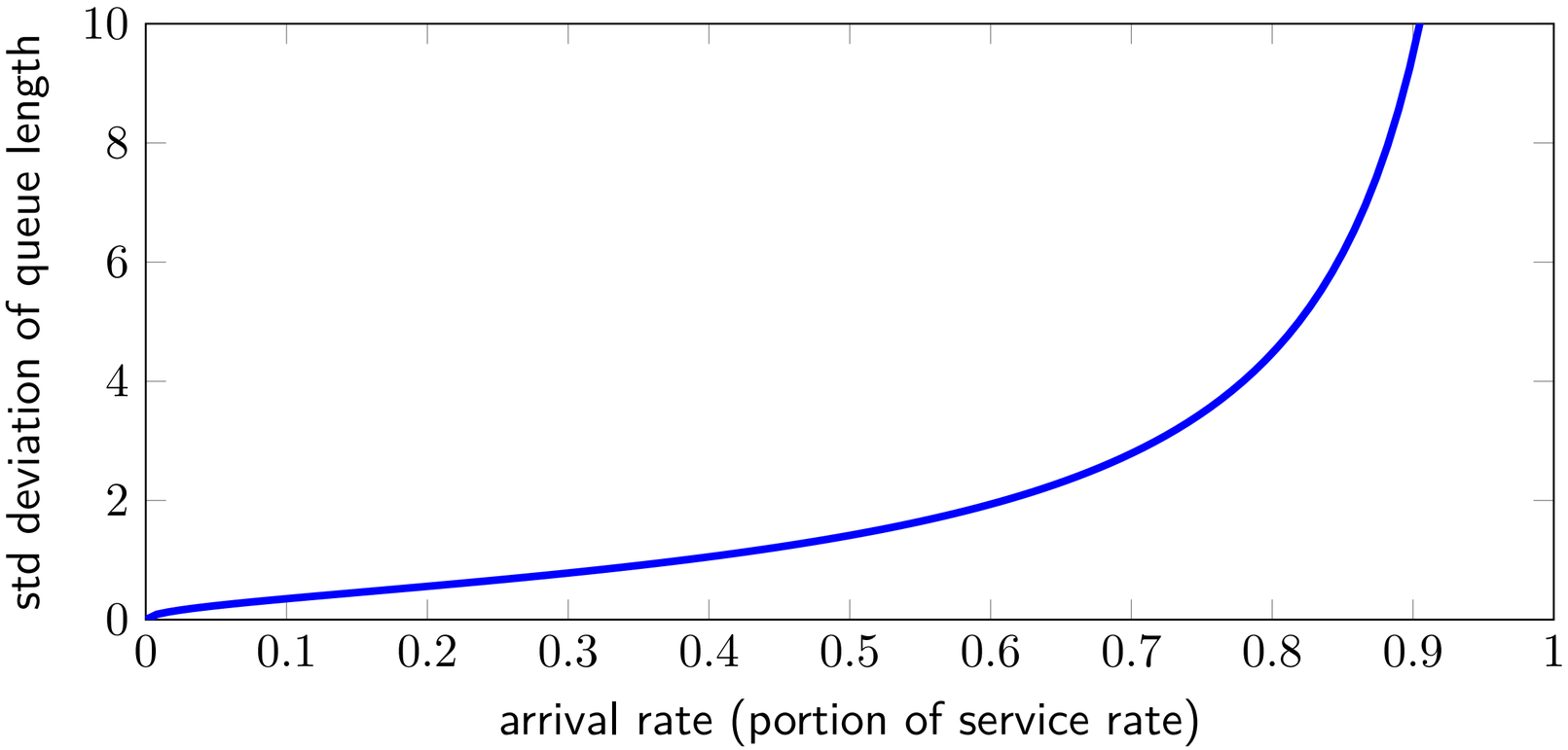


mean queue length

$$\frac{\text{arrival rate}}{\text{service rate} - \text{arrival rate}}$$

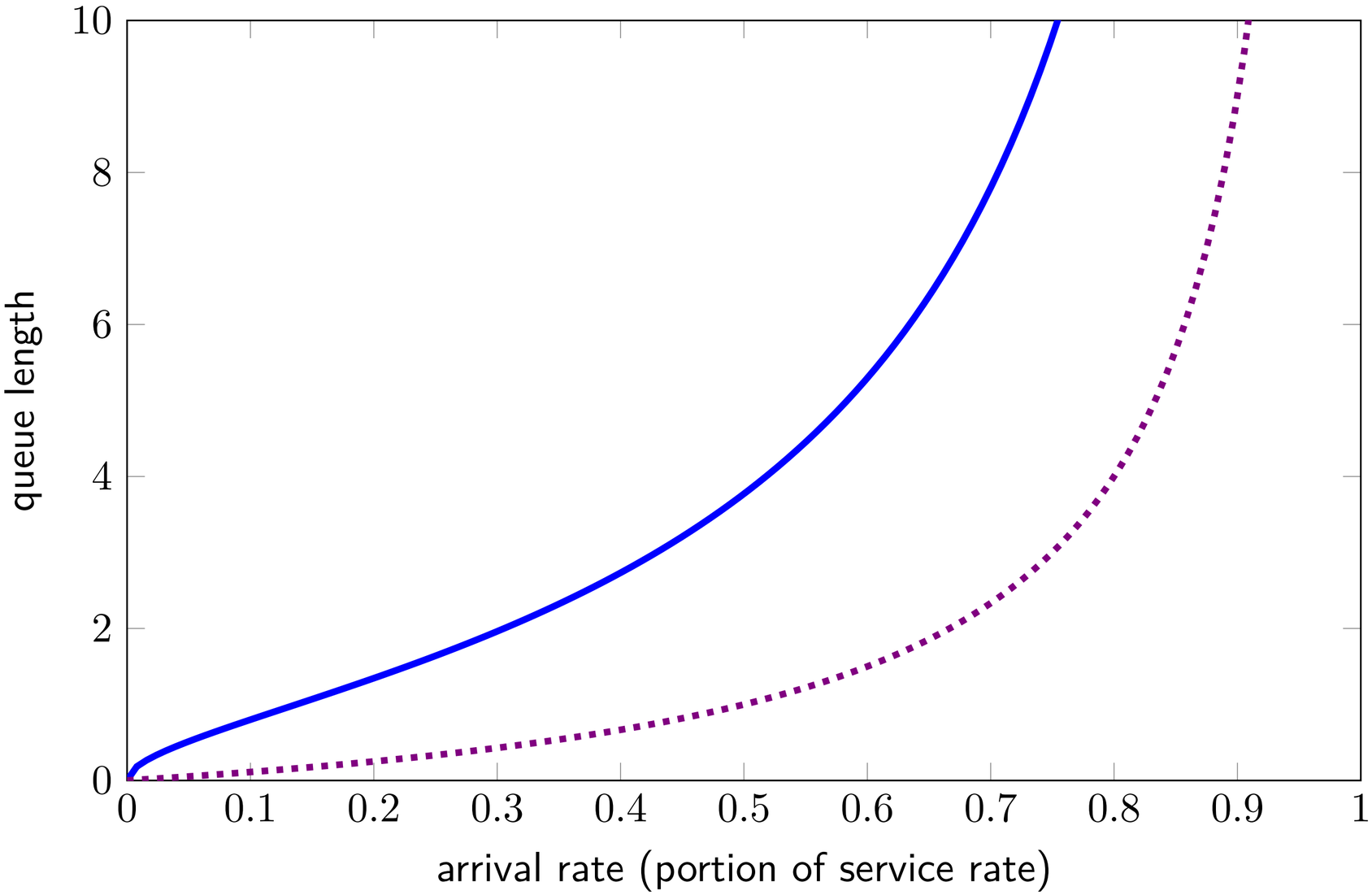
practical implication: *need to run networks at much less than full utilization*

M/M/1/∞ queue length std. deviation

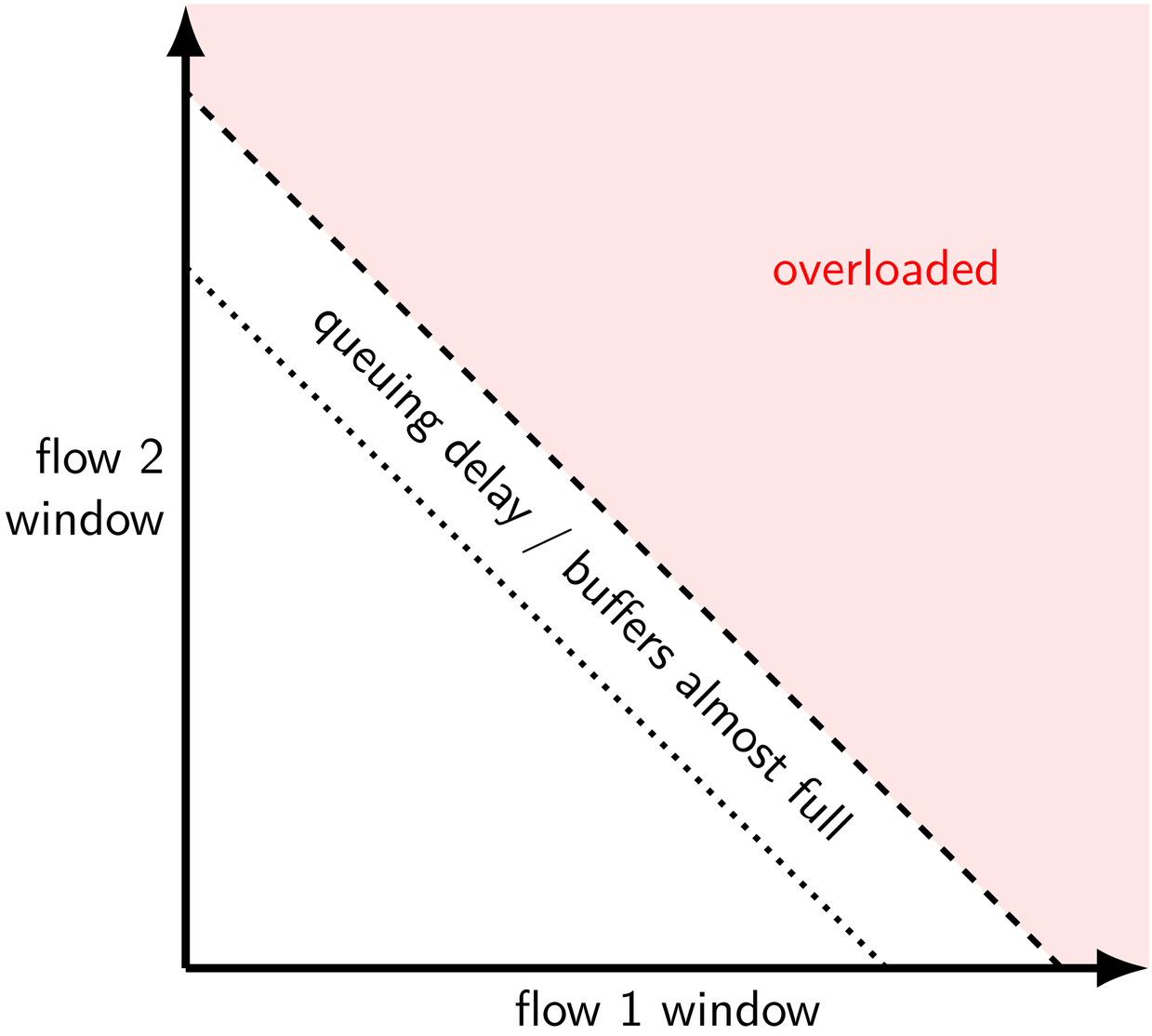


$$\sqrt{\frac{\text{utilization}}{(1-\text{utilization})^2}}$$

approx 95th pctile v mean queue length

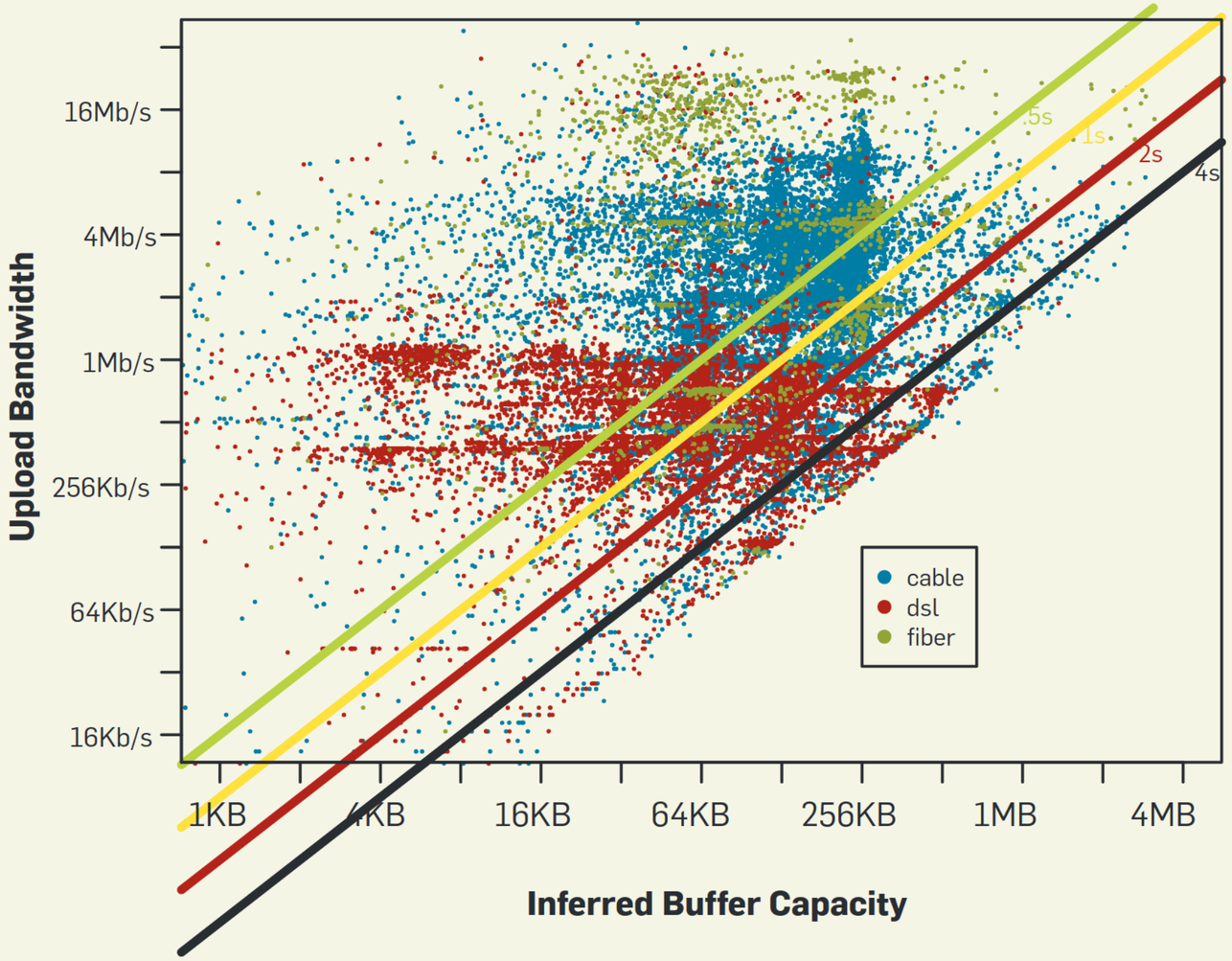


filling buffers



big buffers? (in 2011 or so)

Figure 5. Plot reproduced from ICSI's Netalyzr studies.



Jim Gettys and Kathleen Nichols, "Bufferbloat: Dark Buffers in the Internet" (CACM, Jan 2012)

problems with big buffers

high latency — bad for some applications

slower response to congestion

- 1 second round trip time = 1 second to detect congestion
- more likely to have ‘congestion collapse’

avoiding big buffers

multiple fixes (that can be combined):

use smaller buffers?

- simpliest solution
- but will lower performance if done too aggressively

detect congestion without full buffer…

- by choosing when/which packets to drop better?
- by using something other than drops?

fixes from Jacobson's 1987 paper

1. *round-trip-time variance estimation*
2. exponential retransmit timer backoff
3. slow-start
4. more aggressive receiver ack policy
5. dynamic window sizing on congestion
6. Karn's clamped retransmit backoff
7. fast retransmit

timeout setting

goal in setting timeouts:

- timeout triggering almost always means dropped packet

to do this want *highest likely round trip time*

original TCP heuristic: twice RTT estimate

RTT variation exercise (1)

let’s say 1 ms tranmission delay + 20 ms propogation delay

and queue depth ranges ‘randomly’ from 1 to 10

exercise: round-trip-time?

- 42 ms with no queue
- +1 ms per queue depth
- 43 to 52 ms

RTT variation exercise (2)

let’s say 1 ms tranmission delay + 10 ms propogation delay

and queue depth ranges ‘randomly’ from 10 to 40

exercise: round-trip-time?

- 12 ms with no queue
- +1 ms per queue depth
- 22 to 52 ms

how does original timeout do?

works well when queuing delay small relative to other delays

works poorly when queuing delay high

… because queuing delay won’t be consistent!

new timeout formula

estimate mean deviation of RTT (= difference from average)

similar exponentially weighted moving average

timeout = RTT estimate + 2 $\times$ RTT deviation estimate

fixes from Jacobson's 1987 paper

1. round-trip-time variance estimation
2. *exponential retransmit timer backoff*
3. slow-start
4. more aggressive receiver ack policy
5. dynamic window sizing on congestion
6. Karn's clamped retransmit backoff
7. fast retransmit

normal backoff

problem: what if we have multiple timeouts

let's say timeout is 1 time unit

transmit at 1 time unit, 2 time units, 3 time units, 4 time units, etc.

problem: if the network is overloaded *from retransmissions* won't stop it

- … but window size reduction should make number of packets retransmitted *per connection* low
- (so probably not so important with corrected window size management?)

exponential backoff

instead of:

transmit at 1 time unit, 2 time units, 3 time units, 4 time units, etc.

do something like:
transmit at 1 time unit, 3 time units, 7 time units, 15 time units, etc.

exponential backoff theory

for binary exponential backoff

- timeout for ith retransmission is $2^i \times \text{base timeout}$

intuition: avoids overloading network by being a lot less aggressive

- what you 'should' do for repeated timeouts due to congestion

not aware of good theoretical results in TCP context

- famous result that this type of backoff is good for things like deciding when to retransmit on shared wired/wireless medium
- (Goodman et al, "On Stability of Ethernet")

“traditional” TCP variant names

everything we’ve talked about as standard = NewReno

Tahoe — slow start + redo slow start on any loss + fast retransmit

Reno — Tahoe + halve window size on dup ACKs

NewReno — Reno + fast recovery (send extra during fast retransmit)

SACK — NewReno + use selective acknowledgments

more recent TCP variants

BIC, CUBIC

loss-based schemes that vary increase/decrease formulas

Vegas, BBR, FAST, Compound, Westwood

use latency/bandwidth to detect congestion

(and there are many, many more)

CUBIC: default congestion control today

default in Linux (since 2006), OS X (since 2014), Windows (since 2019)

- sysadmin has other options they can configure
- can be changed on connection-by-connection basis

has non-additive increase

increase intuition

‘standard’ TCP basically has two increase algorithms:

slow start — quickly explore window sizes

congestion avoidance — slowly probe for maximum

abrupt transition between these is suspect

- should have gradual exploration $\rightarrow$ fine-tuning switch

heuristic for when to use slow start-style is suspect

- half way from last loss $\approx$ guess?

CUBIC increase algorithm

big idea: faster increase when further away from window size of last loss

- cubic function with saddle point at that window size
- function parameterized by time since loss

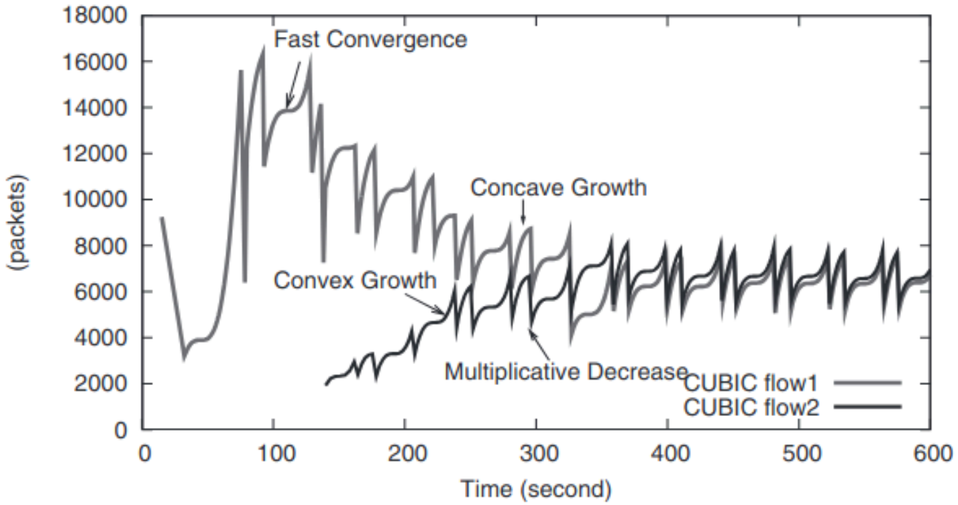
intuition for ‘steady state’:

- guess correct window size is close to last loss
- quickly get to around that window size
- search slowly/precisely around that size to fine tune size

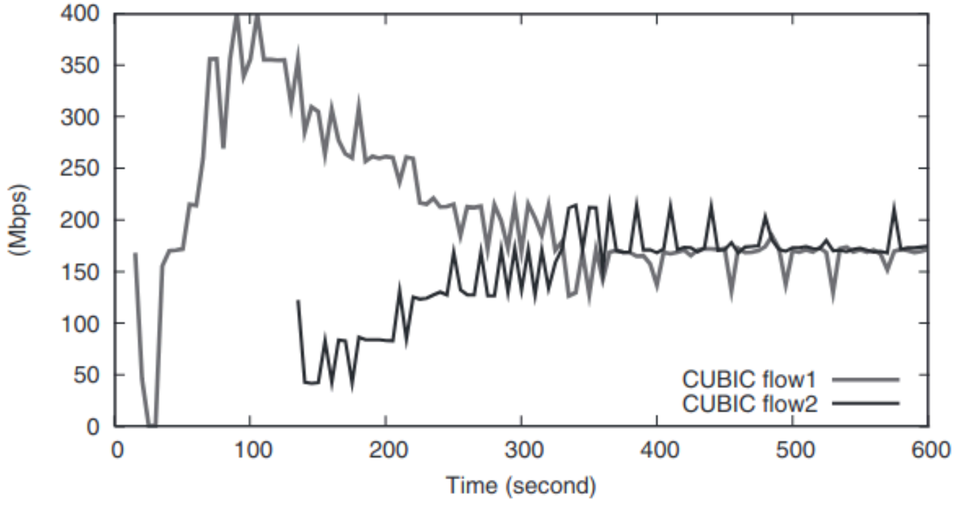
intuition for non-steady state:

- try to quickly probe for roughly how small/big it is
- worry about getting more precise size on later ‘pass’

from Ha, Rhee, and Xu, "CUBIC: A New TCP-Friendly High-Speed TCP Variant"



(a) CUBIC window curves.



(b) Throughput of two CUBIC flows.

Figure 4: Two CUBIC flows with 246ms RTT.

other CUBIC changes

different multiplicative decrease factor (divide by 1.4)

other congestion signals

so far: detecting congestion via drops

- need data to go missing
- transmitting redundant data
- filling up buffers causing high latency

some alternate ideas ways to detect congestion:

have switches/routers ‘mark’ packets

latency from longer queues

other congestion signals

so far: detecting congestion via drops

- need data to go missing
- transmitting redundant data
- filling up buffers causing high latency

some alternate ideas ways to detect congestion:

have switches/routers 'mark' packets

latency from longer queues

ECN

explicit congestion notification

when buffer ‘close’ to full

switches set ‘congestion experienced’ (CE) signal in some packets

goal: congestion signal *instead of* packet drops

- avoid all the retransmission, hopefully

still have fallback to dropping packets

ECN and TCP/IP

congestion experience (CE) signal in IP header

when ACKing, “return” CE signal with ACK

ECN echo (ECE) TCP flag on ACK packets

when sender sees ECE flag, confirm reciept by setting “congestion window reduced” (CWR) flag until ECE flag stops being set

ECN opt-in

two bits in IP header

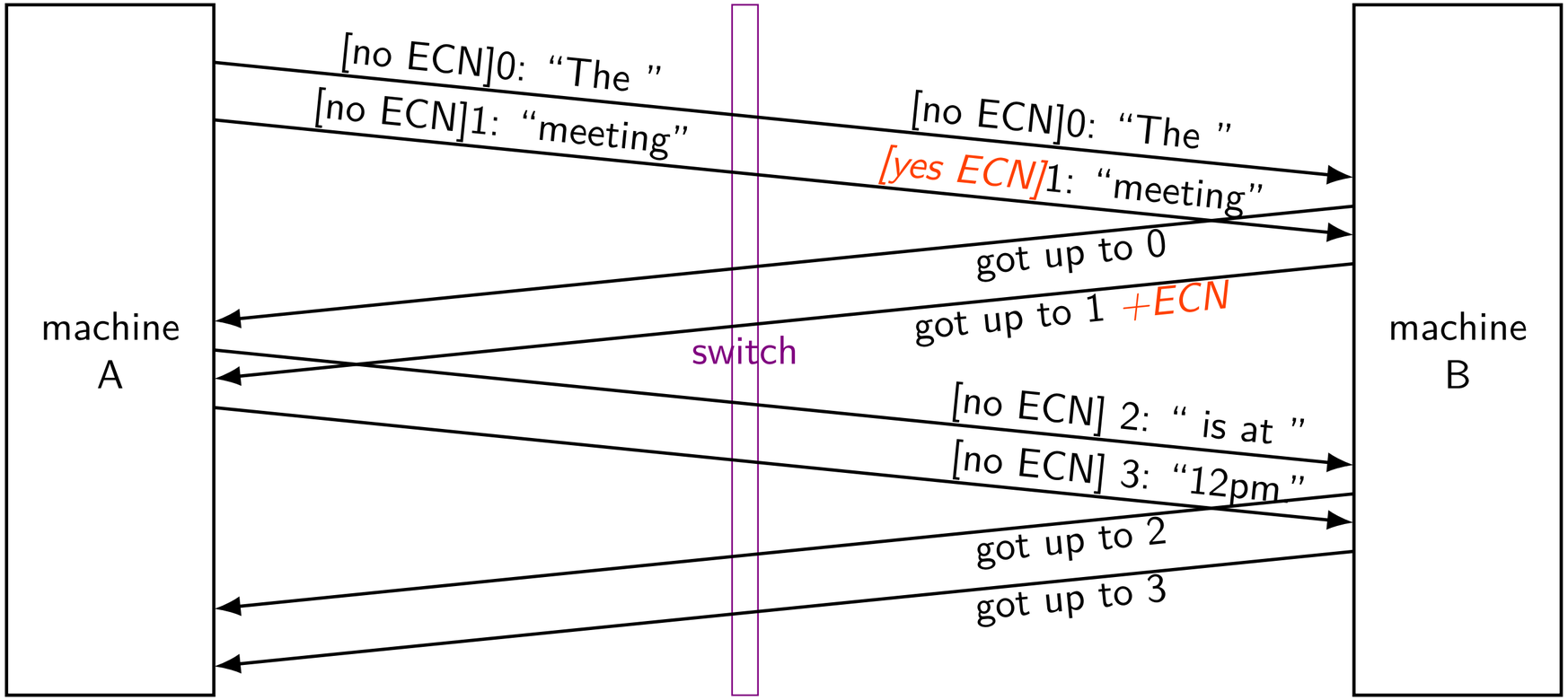
RFC 3168 says:

`00` = not-ECN capable (default)

`01`, `10` = ECN-capable (set by TCP/etc. implementation)

`11` = congestion experienced

ECN timeline



data sent has place for ECN bit to be placed

switch modifies ECN bit *if buffer close to full*

ACK indicates if ECN bit was set

reacting to ECN marks

multiple options for using ECN marks

simplest idea:

adjust window as if packet was dropped

… but don’t need to resend data

ECN deployment

ECN proposed in 2001

13 years later: around 56% support on websites[1]

16 years later:[2]

around 80% support on websites; 0.2% servers broken if ECN requested

20 years later:[3]

around 86% support on websites; 4% of paths strip ECN signals

around 7.5% of connections (from sampled Universities) enable ECN

24 years layer:[4]

around 94% of reachable IPv4/v6 addresses support consistently

around 4% support depends on path taken to host

1. Trammel et al, “Enabling Internet-Wide Deployment of Explicit Congestion Notification”
2. Kühlewind et al, “Tracing Internet Path Transparency”
3. Lim et al, “A Fresh Look at ECN Traversal in the Wild”
4. Zeynali et al, “The State of ECN adoption in the Internet”

example: DCTCP

DataCenter TCP (2010)

intended for datacenters

- high bandwidth, low latency networks

based on explicit congestion notification

… but uses different multiplicative decrease strategy

measure portion of packets marked recently α

decrease by factor of $1 - \alpha/2$

- respond gradually to congestion
- start responding early (packets marked when queues far from full)

other congestion signals

so far: detecting congestion via drops

- need data to go missing
- transmitting redundant data
- filling up buffers causing high latency

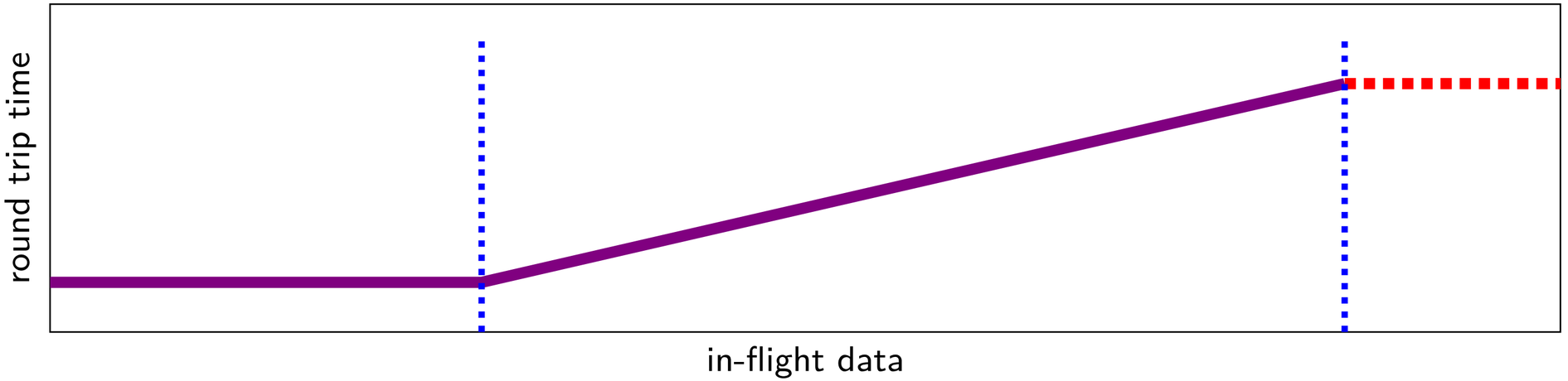
some alternate ideas ways to detect congestion:

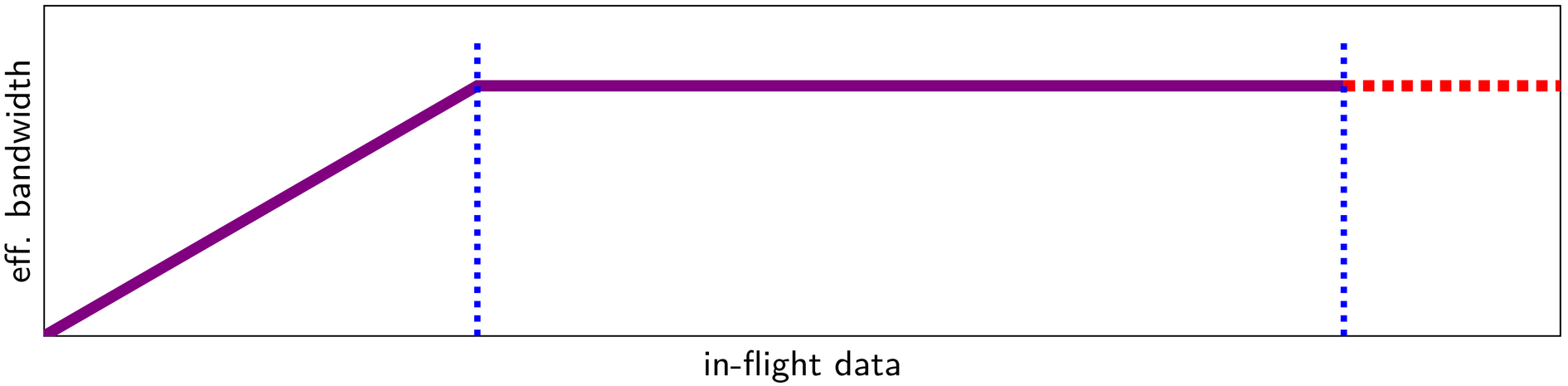
have switches/routers ‘mark’ packets

latency from longer queues

some intuition (based on BBR)

(based on Google's BBR presentation to the IETF)





very different congestion control

fuller queues $\rightarrow$ higher latency

fuller queues $\rightarrow$ throughput same as window increases

strategy: monitor throughput/latency to detect full queues

- goal: fill link without making queue grow (much) in size

‘Vegas’-style congestion control (1)

record “base” round-trip time

- connection start or lowest observed

“ideal” throughput should be one window / base round trip time

- (Vegas paper calls this “expected” throughput)
- what would happen with no queuing delay

“actual” throughput $\approx$ one window / actual round trip time

‘Vegas’-style congestion control (2)

measured ideal+actual throughput (prev. slide)

- mainly using idea of ‘base’ round trip time

goal: control what “ideal” - actual throughput is

if 0, queues are probably empty, can increase window

if large, queues are too big, decrease window

Compound TCP

combines Vegas and ‘normal’ TCP congestion control

track seperate ‘delay’ and ‘congestion’ window

- congestion window uses standard TCP algorithm
- delay window based on Vegas-like increase in RTT detection

effective window size based delay+congestion window

delay-based increase can be multiplicative

was default on Windows for many years

Westwood TCP

Vegas = watch latency versus window size

intuition: if window size $\uparrow \implies$ latency $\uparrow$, window size too high

alternate idea: bandwidth versus window size

intuition: if window size $\uparrow \implies$ bandwidth unchanged, window size too high

Westwood TCP:

estimate actual bandwidth based on ACK rate

set ssthresh based on bandwidth + min RTT

BBR-style congestion control

if queues are empty, larger window:

latency stays the same and throughput increases

if queues are filling, larger window:

throughput stays the same and latency increases

observe effect of sending more/fewer packets periodically

estimate ‘boundary’ based on observed latency/throughput

keep window size near boundary most of the time

BBR-style congestion control

if queues are empty, larger window:

latency stays the same and throughput increases

if queues are filling, larger window:

throughput stays the same and latency increases

observe effect of sending more/fewer packets periodically

estimate ‘boundary’ based on observed latency/throughput

keep window size near boundary most of the time

BBR-style congestion control

if queues are empty, larger window:

latency stays the same and throughput increases

if queues are filling, larger window:

throughput stays the same and latency increases

observe effect of sending more/fewer packets periodically

estimate ‘boundary’ based on observed latency/throughput

keep window size near boundary most of the time

BBR-style congestion control

if queues are empty, larger window:

latency stays the same and throughput increases

if queues are filling, larger window:

throughput stays the same and latency increases

observe effect of sending *more/fewer packets periodically*

estimate ‘boundary’ based on observed latency/throughput

keep window size near boundary most of the time

BBR

congestion control algoirithm out of Google published c. 2016

apparently deployed (at least at some point) on their servers

not great fairness results with traditional TCP

Philip (IMC'21)[1] claims one BBR flow takes 40% of throughput when competing with thousands of CUBIC or NewReno flows

1. Philip, Ware, Athapathu, Sherry, Sekar, "Revisting TCP Congestion Control Throughput Models & Fairness Properties At Scale" (IMC'21)

2023ish measurement data

Table 11: Evolution of the Internet's Congestion Control Landscape (2001–present).

Class	CCA	2001 [54]	2004 [41]	2011 [63]	2019 [50]	2023, *Nebby*
Loss-based AIMD	New Reno [55]	35% (1,571)	25% (21,266)		0.8% (160)	11.1% (11,157)
	Reno [40]	21% (945)	5% (4,115)	12.5% (623)	-	-
	Tahoe	26% (1,211)	3% (2,164)		-	-
Loss-based MIMD	CUBIC [34]	-	-	22.3% (1,115)	30.7% (6,139)	41% (41,085)
	BIC [62]			10.6% (531)	0.9% (181)	3% (2,985)
	HSTCP [28]			7.4% (369)	R	0% (0)
	Scalable [43]			1.4% (69)	0.2% (39)	0% (24)
Delay-based AIMD	Vegas [19]	-	-	1.2% (58)	2.8% (564)	2.6% (2,637)
	Westwood [23]			2% (104)	0% (0)	0.4% (371)
Delay-based MIMD	CTCP [59]	-	-	6.7% (334)	5.7% (1,148)	C
	Illinois [45]			0.6% (28)		3.4% (3,380)
	Veno [29]			0.9% (45)	V	0.6% (578)
	YeAH [18]			1.4% (72)	5.8% (1,162)	0.4% (388)
	HTCP [44]			0.4% (18)	2.8% (560)	2.3% (2,259)
Rate-based	BBRv1 [21]	-	-	-	17.8% (3,550)	10% (9,985)
	BBR G1.1 [50]				0.8% (167)	-
	BBRv2 [22]				-	1.1% (1,151)
	BBRv3				-	0.2% (204)
Unclassified		17.3% (792)	53% (44,950)	4% (198)	12.2% (2,432)	16.7% (16,733)
AkamaiCC		-	-	-	5.5% (1,103)	7.2% (7,117)
Short Flows		-	-	26% (1,300)	7.5% (1,493)	-
Unresponsive		0.7% (30)	14% (11,529)	-	6.5% (1,302)	-
Abnormal SS*		-	-	2.9% (144)	-	-
Total hosts		100% (4,550)	100% (84,394)	100% (5,000)	100% (10,000)	100% (100,000)

R Classified together with New Reno
V Classified together with Vegas
C CTCP has been deprecated in Windows
* Websites identified by CAAI as having Abnormal Slow Starts

Mishra et al, "Keeping an Eye on Congestion Control in the Wild with Nebby" (SIGCOMM '24)

a way to obtain these measurements

Ware et al (2024) technique: estimate queue occupancy

- add artificial slow link on measurement path
- measure number of packets queued over time
- shape of graph indicates which congestion control algorithm

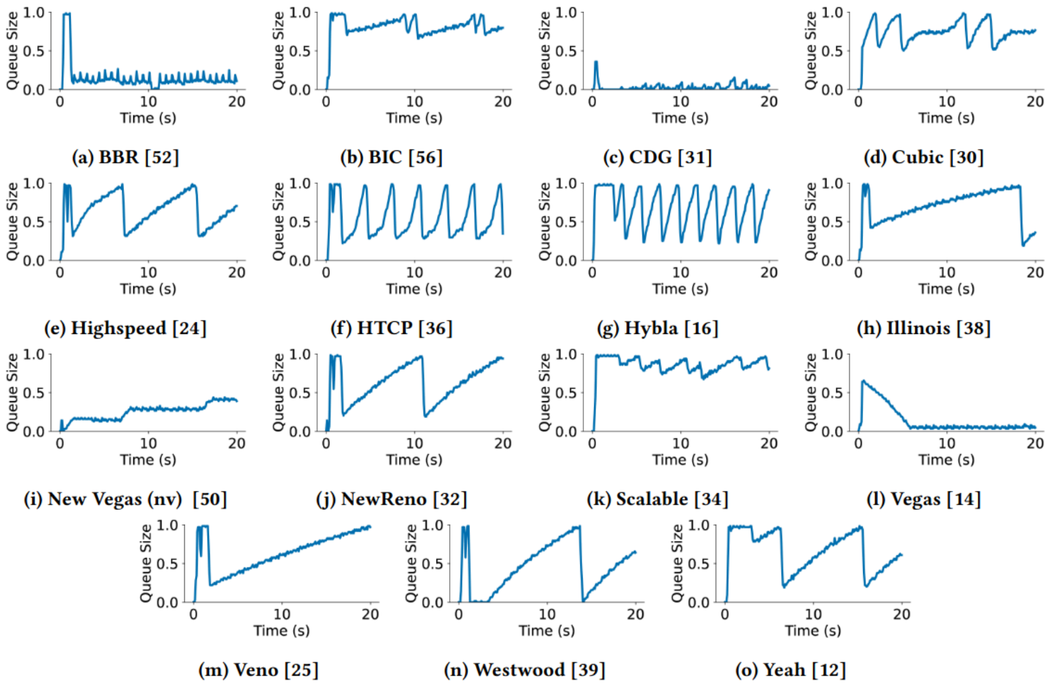


Figure 23: Example CCAnalyzer CCA training sample traces from AWS-Virginia (5bw-85rtt-64q setting)

backup slides

fixes from Jacobson's 1987 paper

1. round-trip-time variance estimation
2. exponential retransmit timer backoff
3. *slow-start*
4. more aggressive receiver ack policy
5. dynamic window sizing on congestion
6. Karn's clamped retransmit backoff
7. fast retransmit

“slow start”

not very well named

exponential(ish) increase to find window size

- used on connection setup or some losses

history: original TCP set high window size

slow start is “slow” because it starts at a small size always

… but it grows *really quickly*

“slow start”

set window size = $\sim$ 2 packet

increase by one packet for each ACK

… until first packet loss

use window size before first loss

slow start effect

suppose we never leave slow start in connection between A and B, with initial window size of 1 packet, and:

A sends 4 packets to B

after receiving 4 packets, B sends 8 packets to A

after receiving those packets A sends 1 packet to B

how many round-trip times does this take?

TCP Tahoe

congestion avoidance (cwnd $\geq$ ssthresh)

no loss: cwnd += 1 segment per round-trip-time

on loss: ssthresh = cwnd/2; cwnd = ~ 2 segments

"slow start" (cwnd < ssthresh)

no loss: cwnd += 1 segment per newly ack'd segment

on loss: ssthresh = cwnd/2; cwnd = cwnd/2;

recovery (known loss)

retransmit packet after third duplicate ACK of previous packet :or timeout

TCP (New)Reno [with slow start]

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

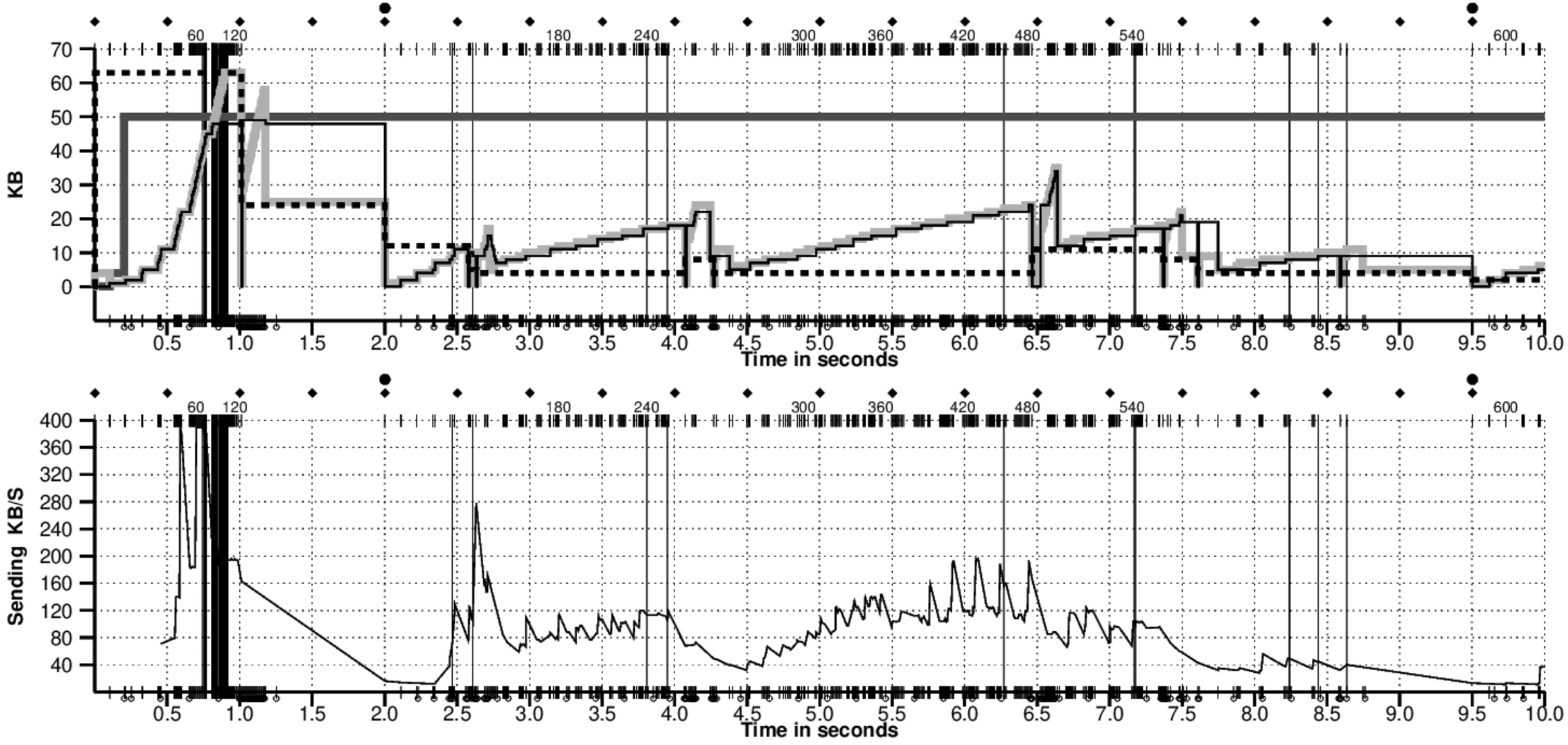
- no loss: cwnd += 1 segment per round-trip-time
- on loss (dup ACK): cwnd = cwnd/2; ssthresh = new cwnd
- on loss (timeout): cwnd = 2 segments

"slow start" (cwnd < ssthresh)

- *no loss: cwnd += 1 segment per newly ack'd segment*
- *on loss: ssthresh = cwnd/2; cwnd = cwnd/2;*

recovery (known loss)

- resend immediately on third dup ACK
- send new data based on self-clocking idea ("fast recovery")

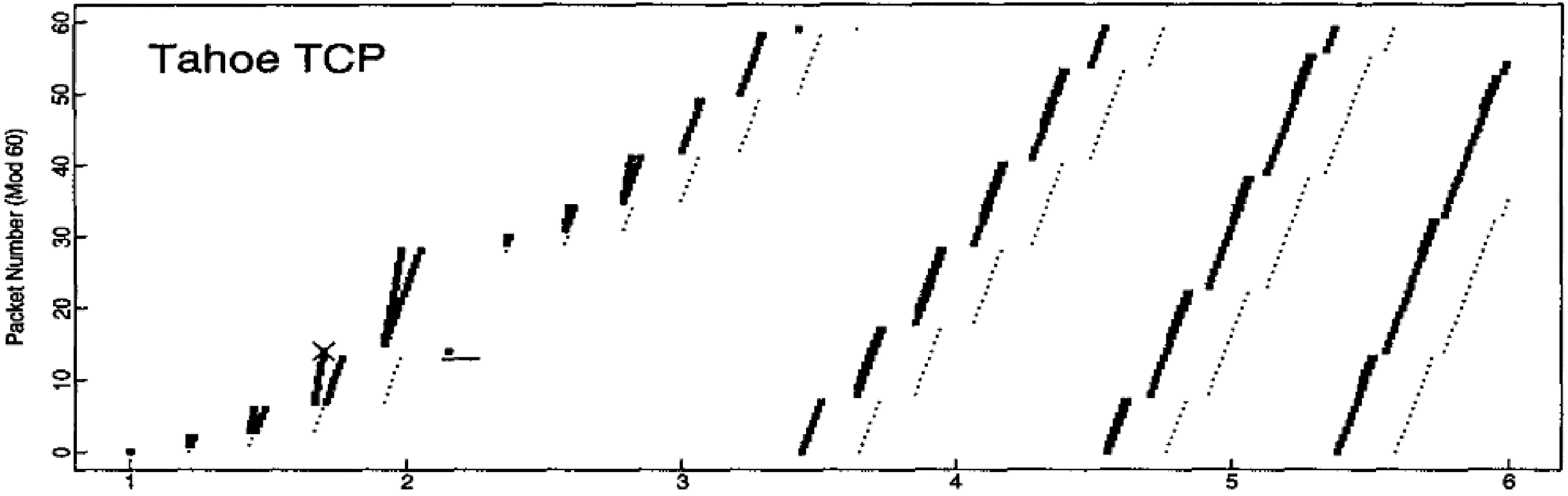


top thick, light-grey line = congestion window; dotted = slow start threshold

from Brakmo, O'Malley, and Peterson, "TCP Vegas: New techniques for congestion detection and avoidance"

TCP 'Tahoe' w/ one loss

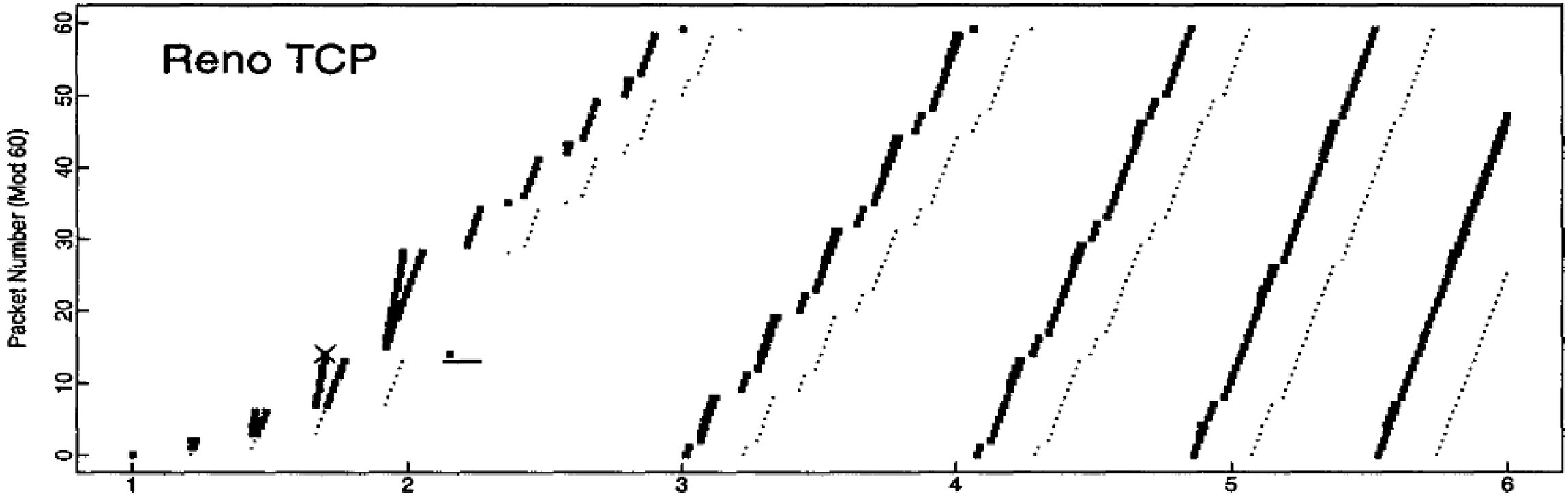
from Kevin Fall and Sally Floyd, "Simulation-based Comparisons of Tahoe, Reno, and SACK TCP"



TCP Tahoe = slow start, fast retransmit, no fast recovery

TCP 'Reno' w/ one loss

from Kevin Fall and Sally Floyd, "Simulation-based Comparisons of Tahoe, Reno, and SACK TCP"



TCP Reno = slow start, fast retransmit/recovery

fixes from Jacobson's 1987 paper

1. round-trip-time variance estimation
2. exponential retransmit timer backoff
3. slow-start
4. more aggressive receiver ack policy
5. dynamic window sizing on congestion
6. Karn's clamped retransmit backoff
7. *fast retransmit*

TCP (New)Reno [with slow start]

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

- no loss: cwnd += 1 segment per round-trip-time
- on loss (dup ACK): cwnd = cwnd/2; ssthresh = new cwnd
- on loss (timeout): cwnd = 2 segments

"slow start" (cwnd < ssthresh)

- no loss: cwnd += 1 segment per newly ack'd segment
- on loss: ssthresh = cwnd/2; cwnd = cwnd/2;

recovery (known loss)

- *resend immediately on third dup ACK*
- send new data based on self-clocking idea ("fast recovery")

fast retransmit

if large window + data packet 2 is lost, then sender will see

ACK 0, *ACK 1, ACK 1, ACK 1, ACK 1, ACK 1*

duplicate ACKs indicate missing packet 2

shouldn't wait for timeout

$\rightarrow$ TCP heuristic: retransmit immediately after ~ 3 duplicate ACKs

not 1 duplicate ACK to tolerate some reordering

also some other details (we'll talk later)

fast retransmission

TCP calls this idea of retransmission from duplicate ACKs "fast retransmission"

means we typically don't wait for timeouts

… assuming just one loss

fast retransmit delays

recv'd	sent
—	data 0-7
ACK 0	
	data 8
ACK 1	
	data 9
ACK 1	
ACK 1	
	data 2
ACK 1	
ACK 1	
ACK 1	
ACK 1	
ACK 1	
ACK 9	
	data 10-…

delayed sending new packets until ACK 9

… but probably could've done that earlier?

TCP (New)Reno [with slow start]

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

- no loss: cwnd += 1 segment per round-trip-time
- on loss (dup ACK): cwnd = cwnd/2; ssthresh = new cwnd
- on loss (timeout): cwnd = 2 segments

"slow start" (cwnd < ssthresh)

- no loss: cwnd += 1 segment per newly ack'd segment
- on loss: ssthresh = cwnd/2; cwnd = cwnd/2;

recovery (known loss)

- resend immediately on third dup ACK
- *send new data based on self-clocking idea ("fast recovery")*

normal packets in flight

recv'd	sent	count of packets in flight
—	data 0-7	8
ACK 0		7
	data 8	8
ACK 1		7
	data 9	8
ACK 2		7
	data 10	8
ACK 3		7
	data 11	8
ACK 4		7
	data 12	8

window size $\sim$ packets in flight normally

duplicate ACK waste

window size $\sim$ packets in flight normally

but duplicate ACKs are exception (say window size 8)

recv'd	sent	count of packets in flight
—	data 0-7	8
ACK 0		7
	data 8	8
ACK 1		7
	data 9	8
ACK 1		7
ACK 1		6
	data 2	5
ACK 1		4
ACK 1		3
ACK 1		2

problem: need to wait until ACK of 2 to make progress

alternate explanation

sender stopped sending while receiving duplicate ACKs

but we know *most messages got there*

means our usage of network doesn't reflect our window size

want to transmit new packets in response to these ACKs

- even though out of 'normal' window

TCP's fast recovery

estimate *packets in flight*

- with no losses, `cwnd` already does this!
- problem: needs adjustment there are losses

transmit new packets when packets in flight < cwnd

- two implementation ideas:
- temporarily 'inflate' cwnd
- track effective send window separately

fast recovery example

recv'd	sent	est. packets in flight	send window size (range)	cwnd = target packets in flight
—	data 0-7	8	8 (0-7)	8
ACK 0		7	8 (0-7)	8
	data 8	8	8 (1-8)	8
ACK 1		7	8 (1-8)	8
	data 9	8	8 (2-9)	8
ACK 1 (+3)		7	8 (2-9)	8
ACK 1 (+3-4)		6	8 (2-9)	8
ACK 1 (+3-5)		5	8 (2-9)	4
	data 2	6	8 (2-9)	4
ACK 1 (+3-6)		5	8 (2-9)	4
ACK 1 (+3-7)		4	8 (2-9)	4
ACK 1 (+3-8)		3	8 (2-9)	4
	data 10	4	9 (2-10)	4
ACK 1 (+3-9)		3	9 (2-10)	4
	data 11	4	10 (2-11)	4

during recovery, "inflate" effective send window size

try to make actual packets in flight match new congestion window size

selective ACKs

selective ACK: ACK 1, plus I got 3–5 and 7–8

can use as another duplicate ACK signal:

example:
resend 6 immediately after
3x “plus I got 3–5 and 7–…”

the reverse path

so far: assuming congestion on sender to receiver path

but we can also have congestion in other direction

- network becomes overloaded with ACKs

hopefully rare because ACKs are small, but…

but worth some special mitigations

delayed ACKs

RFC 1122 (Requirements for Internet Hosts — Communication Layers)

> "A host that is receiving a stream of TCP data segments can increase efficiency… by sending fewer than one ACK (acknowledgment) per data segment received; this is known as a "delayed ACK"… "

usually enabled these days

adds some latency, so Linux lets you disable on per-connection basis

diversion: some queuing theory

queuing theory: applied probability

talks about how queues work

applies to networks and anything else with “waiting in line”

queue measurements

arrival rate

service time (amount of time after waiting in line)

utilization = arrival rate / service time

if single thing can be processed at a time, then max utilization = 100%

- higher implies “infinitely” long queues

M/M/1/∞ queue

next slides: results for M/M/1/∞ queue

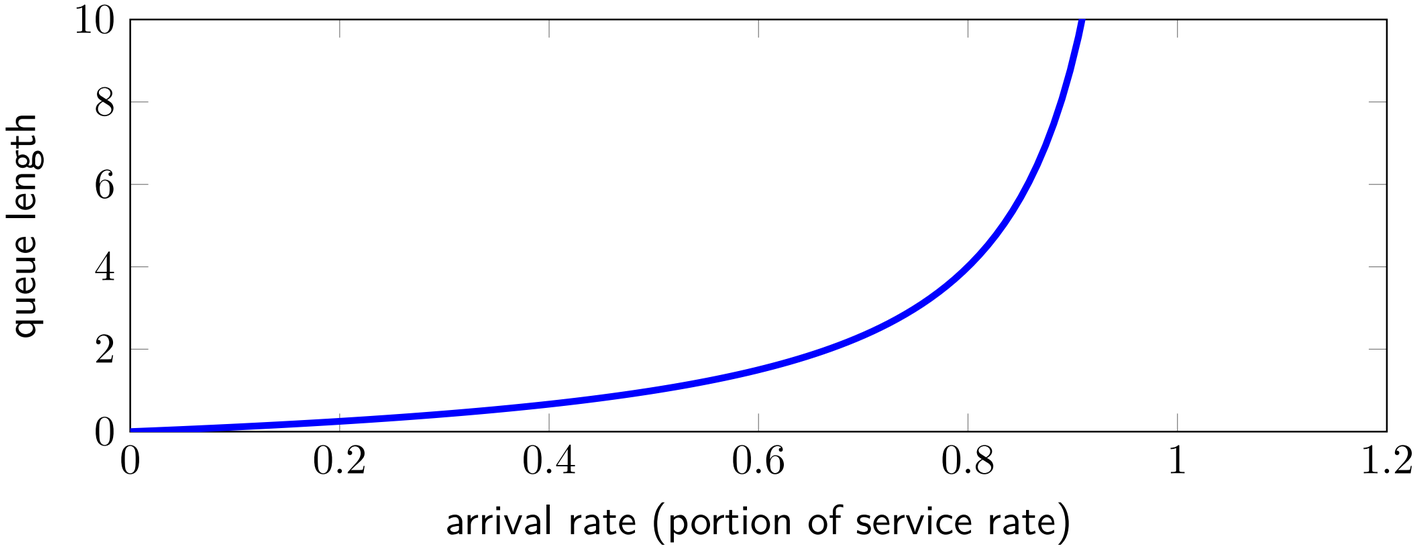
M (memoryless) — random arrival (exponential dist.)

M — random service time (exponential dist.)

1 — one “server” (thing that can process packets)

∞ — unlimited queue length

M/M/1/∞ queue length

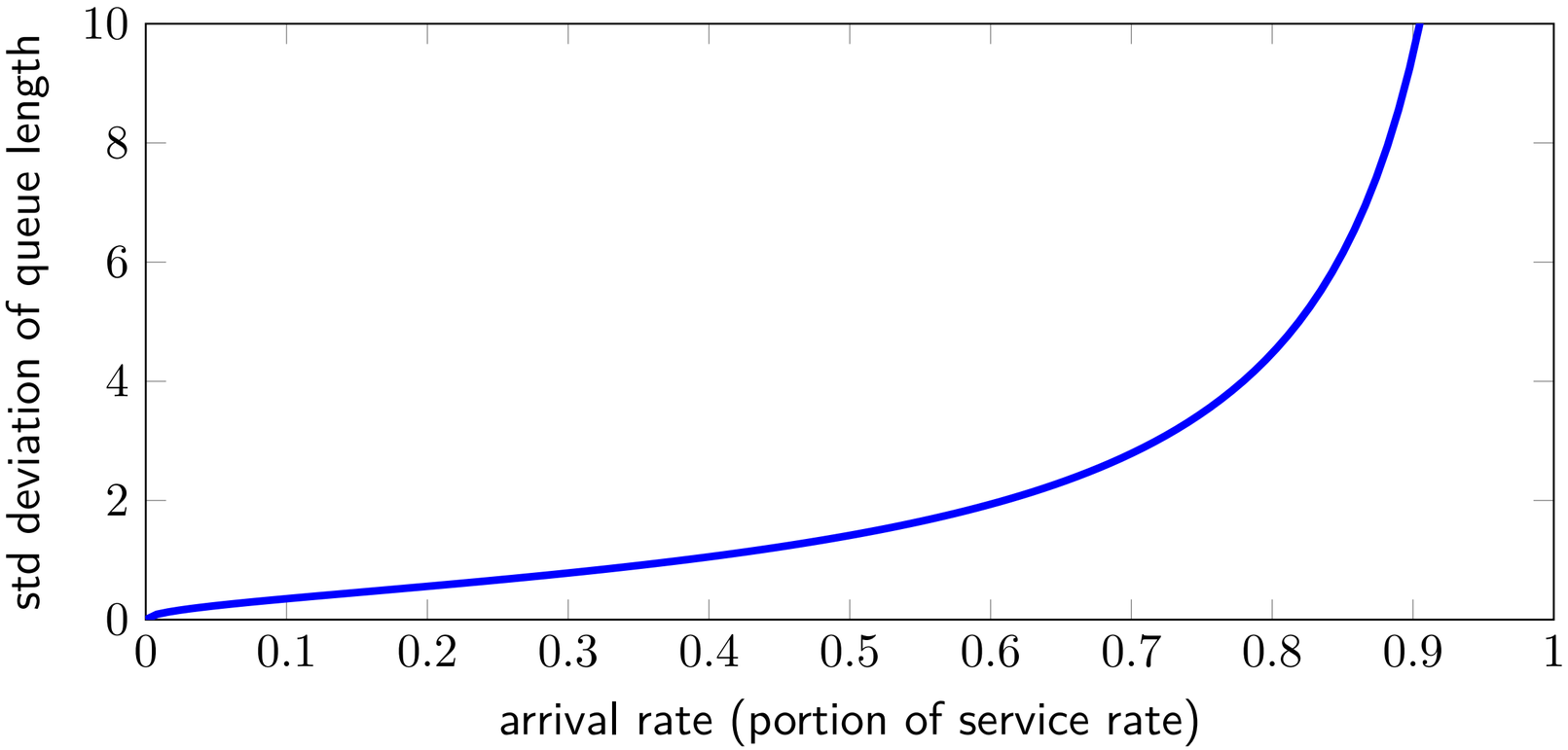


mean queue length

$$\frac{\text{arrival rate}}{\text{service rate} - \text{arrival rate}}$$

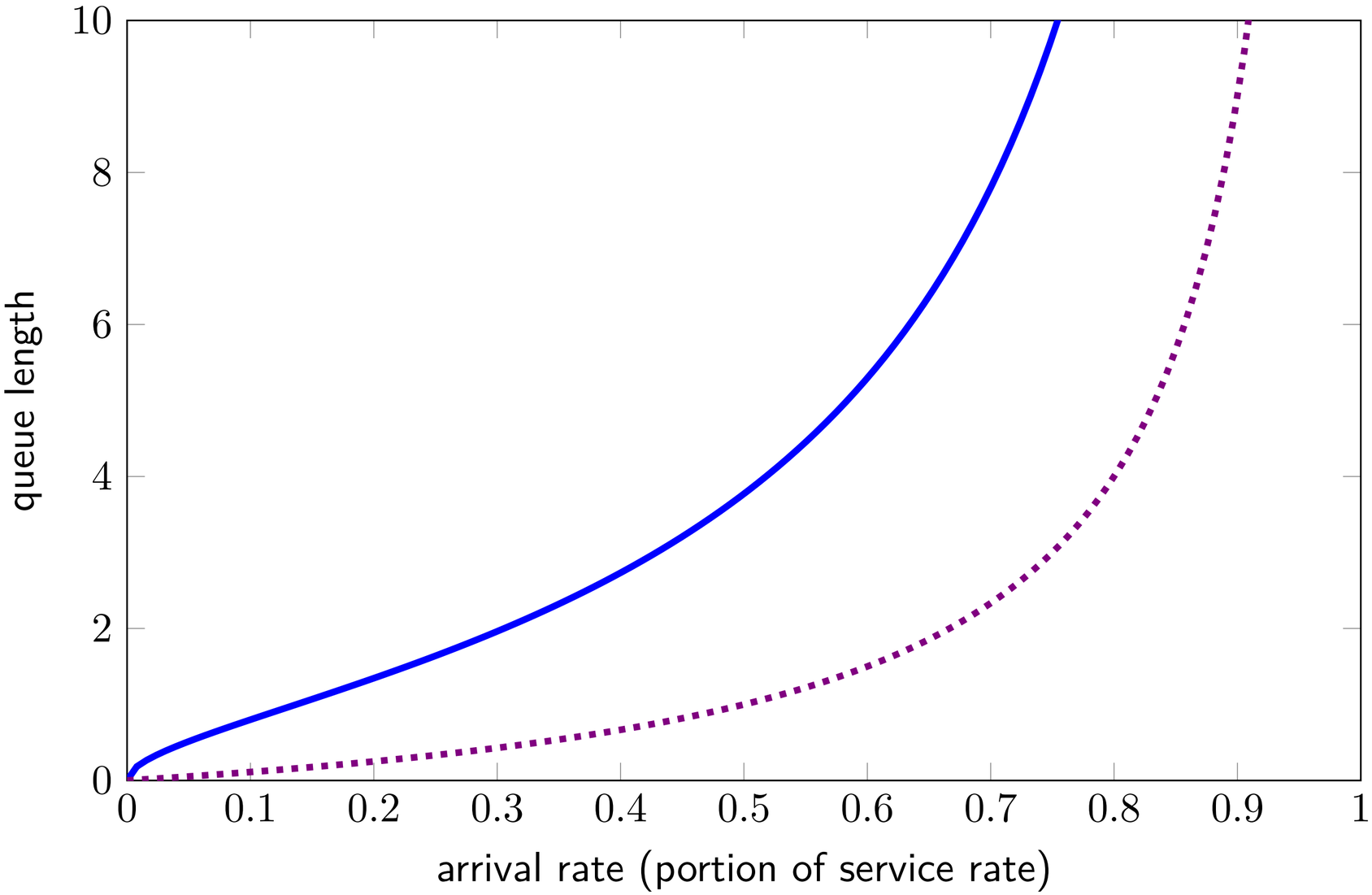
practical implication: *need to run networks at much less than full utilization*

M/M/1/∞ queue length std. deviation

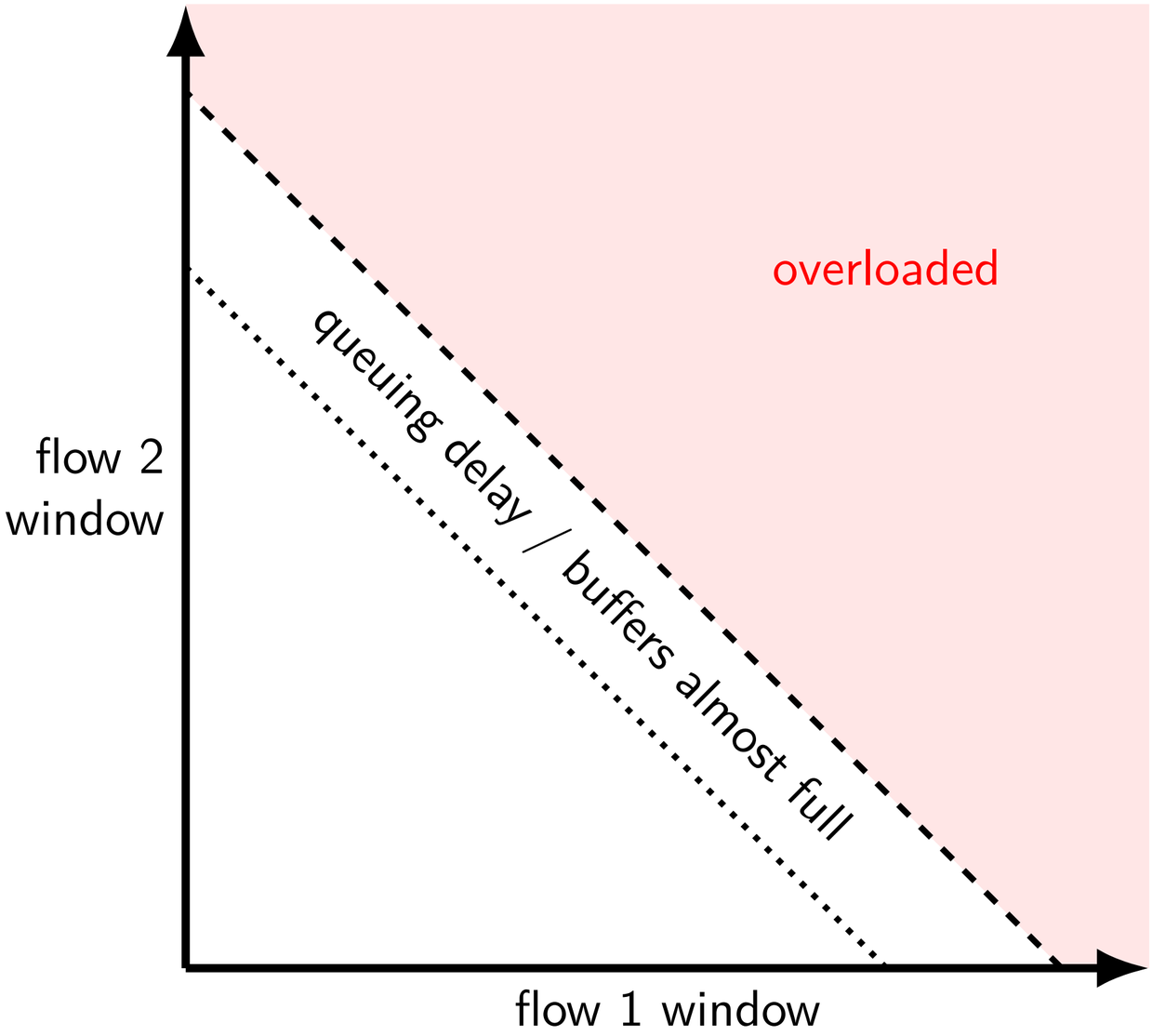


$$\sqrt{\frac{\text{utilization}}{(1-\text{utilization})^2}}$$

approx 95th pctile v mean queue length

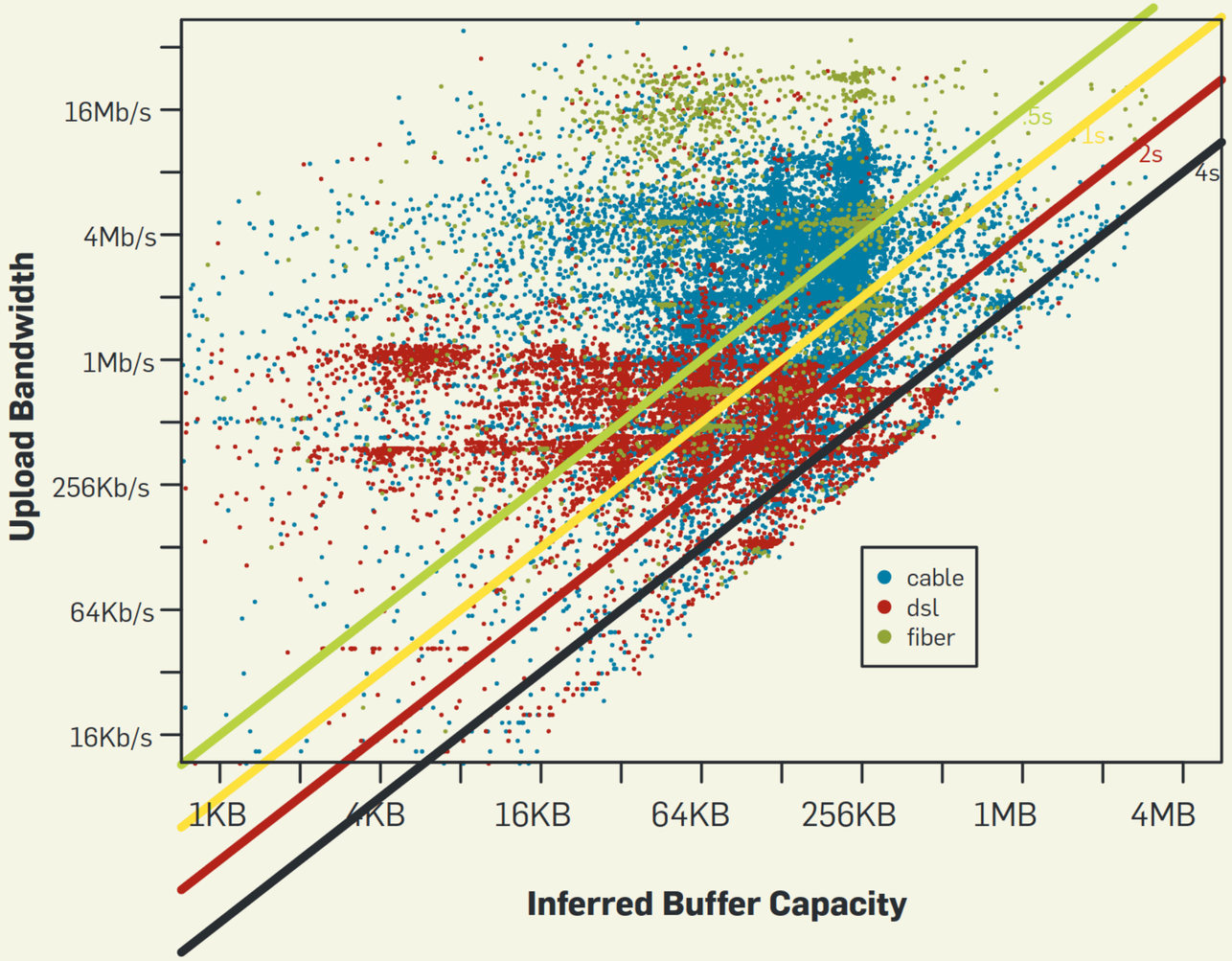


filling buffers



big buffers? (in 2011 or so)

Figure 5. Plot reproduced from ICSI's Netalyzr studies.



Jim Gettys and Kathleen Nichols, "Bufferbloat: Dark Buffers in the Internet" (CACM, Jan 2012)

problems with big buffers

high latency — bad for some applications

slower response to congestion

- 1 second round trip time = 1 second to detect congestion
- more likely to have ‘congestion collapse’

avoiding big buffers

multiple fixes (that can be combined):

use smaller buffers?

- simpliest solution
- but will lower performance if done too aggressively

detect congestion without full buffer…

- by choosing when/which packets to drop better?
- by using something other than drops?

fixes from Jacobson's 1987 paper

1. *round-trip-time variance estimation*
2. exponential retransmit timer backoff
3. slow-start
4. more aggressive receiver ack policy
5. dynamic window sizing on congestion
6. Karn's clamped retransmit backoff
7. fast retransmit

timeout setting

goal in setting timeouts:

timeout triggering almost always means dropped packet

to do this want *highest likely round trip time*

original TCP heuristic: twice RTT estimate

RTT variation exercise (1)

let’s say 1 ms tranmission delay + 20 ms propogation delay

and queue depth ranges ‘randomly’ from 1 to 10

exercise: round-trip-time?

- 42 ms with no queue
- +1 ms per queue depth
- 43 to 52 ms

RTT variation exercise (2)

let’s say 1 ms tranmission delay + 10 ms propogation delay

and queue depth ranges ‘randomly’ from 10 to 40

exercise: round-trip-time?

- 12 ms with no queue
- +1 ms per queue depth
- 22 to 52 ms

how does original timeout do?

works well when queuing delay small relative to other delays

works poorly when queuing delay high

… because queuing delay won’t be consistent!

new timeout formula

estimate mean deviation of RTT (= difference from average)

similar exponentially weighted moving average

timeout = RTT estimate + 2 $\times$ RTT deviation estimate

fixes from Jacobson's 1987 paper

1. round-trip-time variance estimation
2. *exponential retransmit timer backoff*
3. slow-start
4. more aggressive receiver ack policy
5. dynamic window sizing on congestion
6. Karn's clamped retransmit backoff
7. fast retransmit

normal backoff

problem: what if we have multiple timeouts

let's say timeout is 1 time unit

transmit at 1 time unit, 2 time units, 3 time units, 4 time units, etc.

problem: if the network is overloaded *from retransmissions* won't stop it

- … but window size reduction should make number of packets retransmitted *per connection* low
- (so probably not so important with corrected window size management?)

exponential backoff

instead of:

transmit at 1 time unit, 2 time units, 3 time units, 4 time units, etc.

do something like:
transmit at 1 time unit, 3 time units, 7 time units, 15 time units, etc.

exponential backoff theory

for binary exponential backoff

- timeout for ith retransmission is $2^i \times \text{base timeout}$

intuition: avoids overloading network by being a lot less aggressive

- what you 'should' do for repeated timeouts due to congestion

not aware of good theoretical results in TCP context

- famous result that this type of backoff is good for things like deciding when to retransmit on shared wired/wireless medium
- (Goodman et al, "On Stability of Ethernet")

“traditional” TCP variant names

everything we’ve talked about as standard = NewReno

Tahoe — slow start + redo slow start on any loss + fast retransmit

Reno — Tahoe + halve window size on dup ACKs

NewReno — Reno + fast recovery (send extra during fast retransmit)

SACK — NewReno + use selective acknowledgments

more recent TCP variants

BIC, CUBIC

loss-based schemes that vary increase/decrease formulas

Vegas, BBR, FAST, Compound, Westwood

use latency/bandwidth to detect congestion

(and there are many, many more)

CUBIC: default congestion control today

default in Linux (since 2006), OS X (since 2014), Windows (since 2019)

- sysadmin has other options they can configure
- can be changed on connection-by-connection basis

has non-additive increase

increase intuition

‘standard’ TCP basically has two increase algorithms:

slow start — quickly explore window sizes

congestion avoidance — slowly probe for maximum

abrupt transition between these is suspect

- should have gradual exploration $\rightarrow$ fine-tuning switch

heuristic for when to use slow start-style is suspect

- half way from last loss $\approx$ guess?

CUBIC increase algorithm

big idea: faster increase when further away from window size of last loss

- cubic function with saddle point at that window size
- function parameterized by time since loss

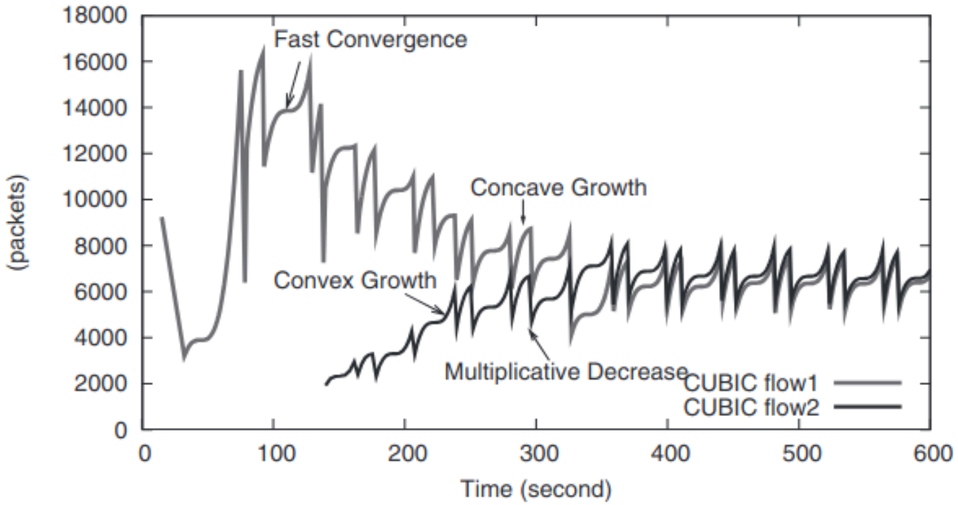
intuition for ‘steady state’:

- guess correct window size is close to last loss
- quickly get to around that window size
- search slowly/precisely around that size to fine tune size

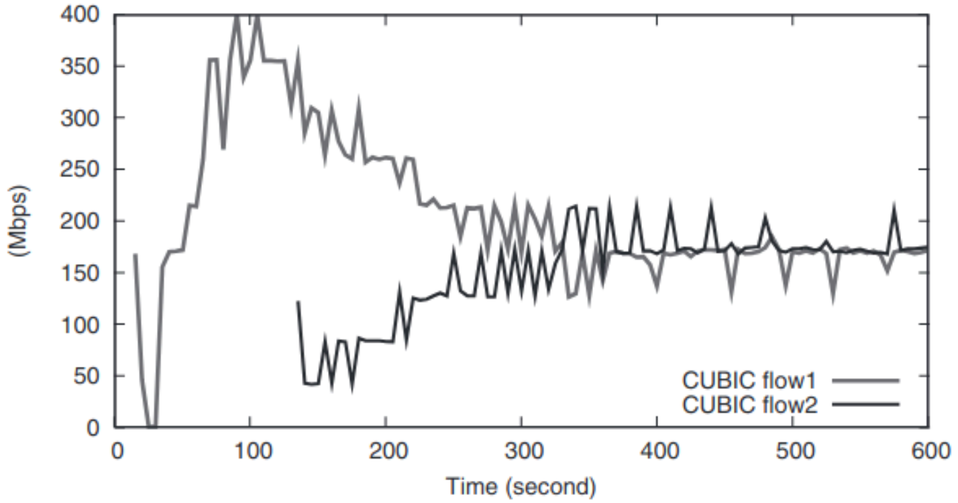
intuition for non-steady state:

- try to quickly probe for roughly how small/big it is
- worry about getting more precise size on later ‘pass’

from Ha, Rhee, and Xu, "CUBIC: A New TCP-Friendly High-Speed TCP Variant"



(a) CUBIC window curves.



(b) Throughput of two CUBIC flows.

Figure 4: Two CUBIC flows with 246ms RTT.

other CUBIC changes

different multiplicative decrease factor (divide by 1.4)

other congestion signals

so far: detecting congestion via drops

- need data to go missing
- transmitting redundant data
- filling up buffers causing high latency

some alternate ideas ways to detect congestion:

have switches/routers ‘mark’ packets

latency from longer queues

other congestion signals

so far: detecting congestion via drops

- need data to go missing
- transmitting redundant data
- filling up buffers causing high latency

some alternate ideas ways to detect congestion:

have switches/routers 'mark' packets

latency from longer queues

ECN

explicit congestion notification

when buffer ‘close’ to full

switches set ‘congestion experienced’ (CE) signal in some packets

goal: congestion signal *instead of* packet drops

avoid all the retransmission, hopefully

still have fallback to dropping packets

ECN and TCP/IP

congestion experience (CE) signal in IP header

when ACKing, “return” CE signal with ACK

- ECN echo (ECE) TCP flag on ACK packets

when sender sees ECE flag, confirm reciept by setting “congestion window reduced” (CWR) flag until ECE flag stops being set

ECN opt-in

two bits in IP header

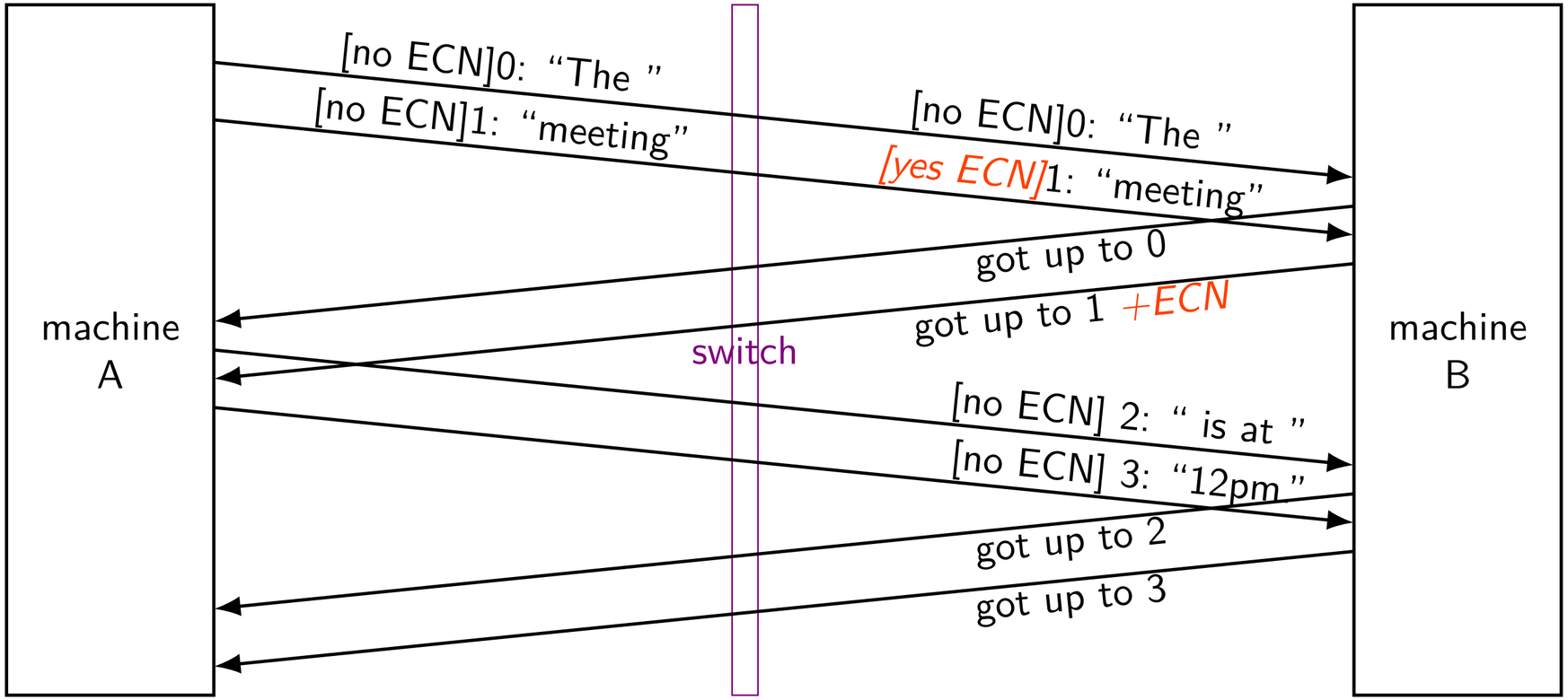
RFC 3168 says:

`00` = not-ECN capable (default)

`01`, `10` = ECN-capable (set by TCP/etc. implementation)

`11` = congestion experienced

ECN timeline



data sent has place for ECN bit to be placed

switch modifies ECN bit *if buffer close to full*

ACK indicates if ECN bit was set

reacting to ECN marks

multiple options for using ECN marks

simplest idea:

adjust window as if packet was dropped

… but don’t need to resend data

ECN deployment

ECN proposed in 2001

13 years later: around 56% support on websites[1]

16 years later:[2]

around 80% support on websites; 0.2% servers broken if ECN requested

20 years later:[3]

around 86% support on websites; 4% of paths strip ECN signals

around 7.5% of connections (from sampled Universities) enable ECN

24 years layer:[4]

around 94% of reachable IPv4/v6 addresses support consistently

around 4% support depends on path taken to host

1. Trammel et al, “Enabling Internet-Wide Deployment of Explicit Congestion Notification”
2. Kühlewind et al, “Tracing Internet Path Transparency”
3. Lim et al, “A Fresh Look at ECN Traversal in the Wild”
4. Zeynali et al, “The State of ECN adoption in the Internet”

example: DCTCP

DataCenter TCP (2010)

intended for datacenters

- high bandwidth, low latency networks

based on explicit congestion notification

… but uses different multiplicative decrease strategy

measure portion of packets marked recently α

decrease by factor of $1 - \alpha/2$

- respond gradually to congestion
- start responding early (packets marked when queues far from full)

other congestion signals

so far: detecting congestion via drops

- need data to go missing
- transmitting redundant data
- filling up buffers causing high latency

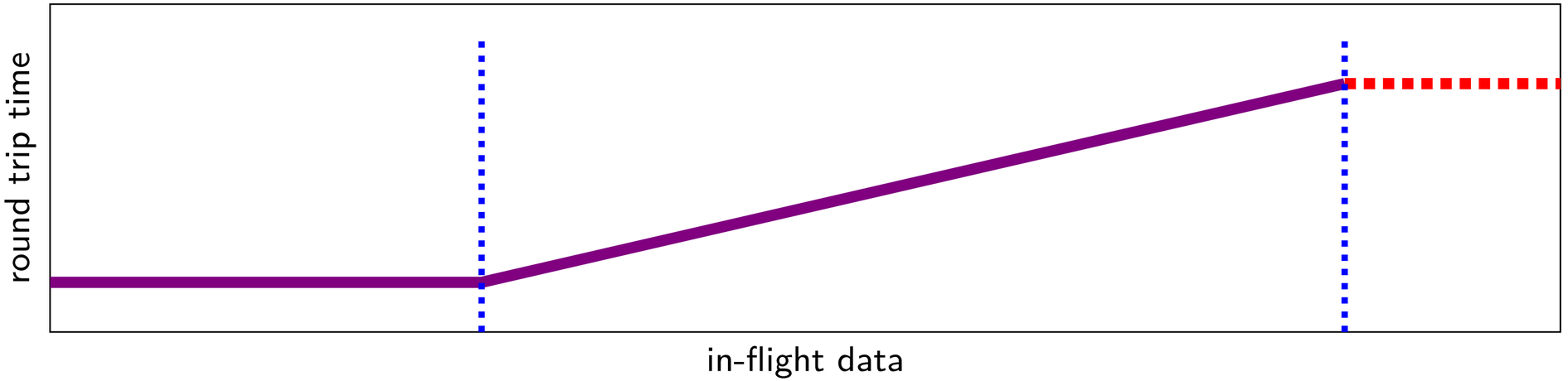
some alternate ideas ways to detect congestion:

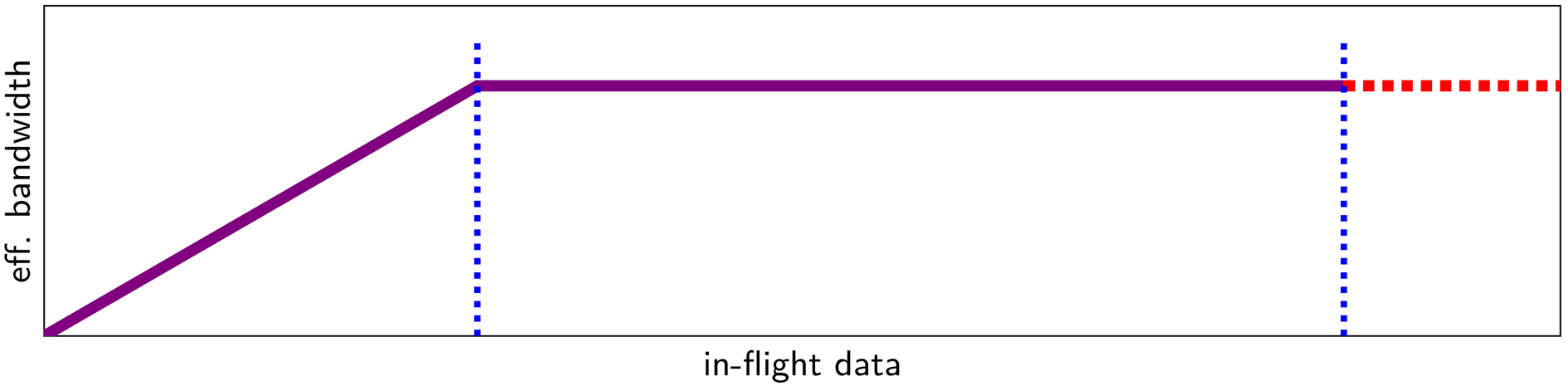
have switches/routers ‘mark’ packets

latency from longer queues

some intuition (based on BBR)

(based on Google's BBR presentation to the IETF)





very different congestion control

fuller queues $\rightarrow$ higher latency

fuller queues $\rightarrow$ throughput same as window increases

strategy: monitor throughput/latency to detect full queues

- goal: fill link without making queue grow (much) in size

‘Vegas’-style congestion control (1)

record “base” round-trip time

- connection start or lowest observed

“ideal” throughput should be one window / base round trip time

- (Vegas paper calls this “expected” throughput)
- what would happen with no queuing delay

“actual” throughput $\approx$ one window / actual round trip time

‘Vegas’-style congestion control (2)

measured ideal+actual throughput (prev. slide)

- mainly using idea of ‘base’ round trip time

goal: control what “ideal” - actual throughput is

if 0, queues are probably empty, can increase window

if large, queues are too big, decrease window

Compound TCP

combines Vegas and ‘normal’ TCP congestion control

track seperate ‘delay’ and ‘congestion’ window

- congestion window uses standard TCP algorithm
- delay window based on Vegas-like increase in RTT detection

effective window size based delay+congestion window

delay-based increase can be multiplicative

was default on Windows for many years

Westwood TCP

Vegas = watch latency versus window size

intuition: if window size $\uparrow \implies$ latency $\uparrow$, window size too high

alternate idea: bandwidth versus window size

intuition: if window size $\uparrow \implies$ bandwidth unchanged, window size too high

Westwood TCP:

estimate actual bandwidth based on ACK rate

set ssthresh based on bandwidth + min RTT

BBR-style congestion control

if queues are empty, larger window:

latency stays the same and throughput increases

if queues are filling, larger window:

throughput stays the same and latency increases

observe effect of sending more/fewer packets periodically

estimate ‘boundary’ based on observed latency/throughput

keep window size near boundary most of the time

BBR-style congestion control

if queues are empty, larger window:

latency stays the same and throughput increases

if queues are filling, larger window:

throughput stays the same and latency increases

observe effect of sending more/fewer packets periodically

estimate ‘boundary’ based on observed latency/throughput

keep window size near boundary most of the time

BBR-style congestion control

if queues are empty, larger window:

latency stays the same and throughput increases

if queues are filling, larger window:

throughput stays the same and latency increases

observe effect of sending more/fewer packets periodically

estimate ‘boundary’ based on observed latency/throughput

keep window size near boundary most of the time

BBR-style congestion control

if queues are empty, larger window:

latency stays the same and throughput increases

if queues are filling, larger window:

throughput stays the same and latency increases

observe effect of sending *more/fewer packets periodically*

estimate ‘boundary’ based on observed latency/throughput

keep window size near boundary most of the time

BBR

congestion control algoirithm out of Google published c. 2016

apparently deployed (at least at some point) on their servers

not great fairness results with traditional TCP

Philip (IMC'21)[1] claims one BBR flow takes 40% of throughput when competing with thousands of CUBIC or NewReno flows

1. Philip, Ware, Athapathu, Sherry, Sekar, "Revisting TCP Congestion Control Throughput Models & Fairness Properties At Scale" (IMC'21)

2023ish measurement data

Table 11: Evolution of the Internet's Congestion Control Landscape (2001–present).

Class	CCA	2001 [54]	2004 [41]	2011 [63]	2019 [50]	2023, *Nebby*
Loss-based AIMD	New Reno [55]	35% (1,571)	25% (21,266)		0.8% (160)	11.1% (11,157)
	Reno [40]	21% (945)	5% (4,115)	12.5% (623)	-	-
	Tahoe	26% (1,211)	3% (2,164)		-	-
Loss-based MIMD	CUBIC [34]	-	-	22.3% (1,115)	30.7% (6,139)	41% (41,085)
	BIC [62]			10.6% (531)	0.9% (181)	3% (2,985)
	HSTCP [28]			7.4% (369)	R	0% (0)
	Scalable [43]			1.4% (69)	0.2% (39)	0% (24)
Delay-based AIMD	Vegas [19]	-	-	1.2% (58)	2.8% (564)	2.6% (2,637)
	Westwood [23]			2% (104)	0% (0)	0.4% (371)
Delay-based MIMD	CTCP [59]	-	-	6.7% (334)	5.7% (1,148)	C
	Illinois [45]			0.6% (28)		3.4% (3,380)
	Veno [29]			0.9% (45)	V	0.6% (578)
	YeAH [18]			1.4% (72)	5.8% (1,162)	0.4% (388)
	HTCP [44]			0.4% (18)	2.8% (560)	2.3% (2,259)
Rate-based	BBRv1 [21]	-	-	-	17.8% (3,550)	10% (9,985)
	BBR G1.1 [50]				0.8% (167)	-
	BBRv2 [22]				-	1.1% (1,151)
	BBRv3				-	0.2% (204)
Unclassified		17.3% (792)	53% (44,950)	4% (198)	12.2% (2,432)	16.7% (16,733)
AkamaiCC		-	-	-	5.5% (1,103)	7.2% (7,117)
Short Flows		-	-	26% (1,300)	7.5% (1,493)	-
Unresponsive		0.7% (30)	14% (11,529)	-	6.5% (1,302)	-
Abnormal SS*		-	-	2.9% (144)	-	-
Total hosts		100% (4,550)	100% (84,394)	100% (5,000)	100% (10,000)	100% (100,000)

R Classified together with New Reno
V Classified together with Vegas
C CTCP has been deprecated in Windows
* Websites identified by CAAI as having Abnormal Slow Starts

Mishra et al, "Keeping an Eye on Congestion Control in the Wild with Nebby" (SIGCOMM '24)

a way to obtain these measurements

Ware et al (2024) technique: estimate queue occupancy

- add artificial slow link on measurement path
- measure number of packets queued over time
- shape of graph indicates which congestion control algorithm

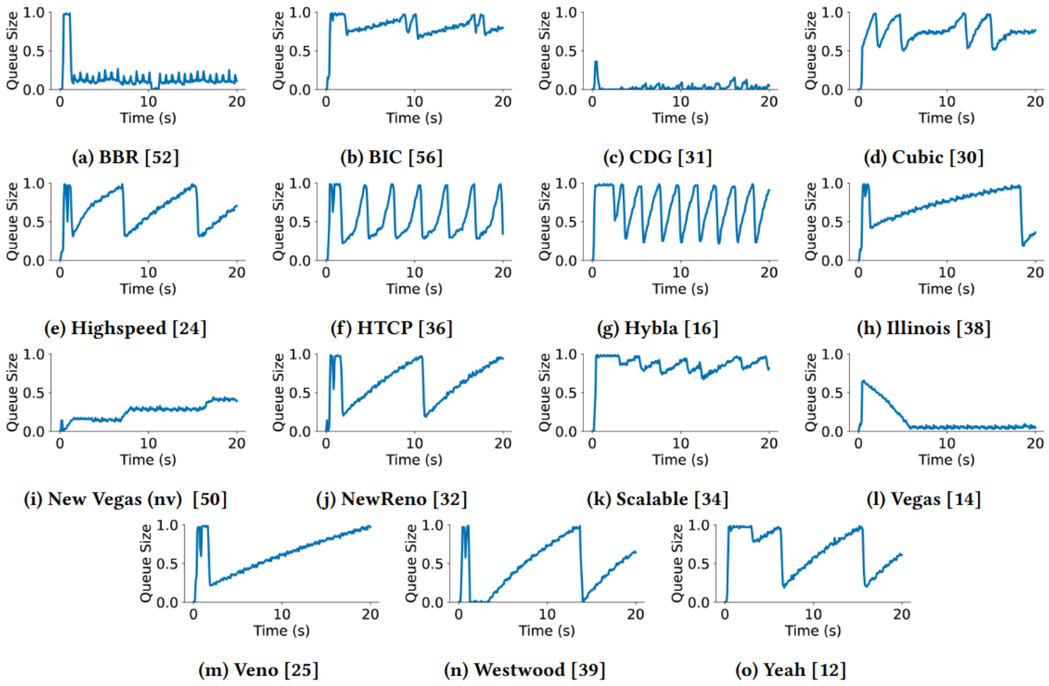


Figure 23: Example CCAnalyzer CCA training sample traces from AWS-Virginia (5bw-85rtt-64q setting)